

Agent 通俗指南

让 AI 从"被动回答"变成"主动干活"——从工具到队友的进化

 = 该术语在 [01_AI_terminology_explanations.md](#) 中有独立词条，可对照阅读

 **本章定位：**学完第 5 章（提示词工程）和第 6 章（Skill 设计）后，你已经会"跟 AI 说话"和"打包技能"了。本章要让 AI **自己判断什么时候用什么技能**——从"你定规矩，AI 照着做"进化到"AI 自己判断，自己干活"。这是从"工具"到"队友"的关键一步。

目录

〇、学完本章你能做什么

- [〇、学完本章你能做什么](#) — 3 个核心能力

一、从一个场景开始

- [一、从一个场景开始](#) — 为什么需要 Agent?

二、Agent 是什么

- [二、Agent 是什么](#) — 定义 + 核心特征 + 类比
 - [2.1 通俗理解](#)
 - [2.2 核心特征](#)

三、能力阶梯

- [三、能力阶梯：从 Prompt 到 Agent](#) — 三级进阶

四、开发范式

- [四、主流开发范式](#) — 4 种范式对比
 - [4.1 简单 LLM 应用](#)
 - [4.2 单智能体](#)
 - [4.3 工作流](#)
 - [4.4 多智能体系统](#)
 - [4.4.1 多 Agent 协作](#) — 为什么需要"团队作战"
 - [4.4.2 通信机制](#) — 中枢 / 链式 / 广播 / 点对点

五、Agent 的组成

- [五、Agent 的组成部分](#) — LLM + 工具 + 记忆 + 规划

六、工作原理

- [六、Agent 是怎么工作的?](#) — 思考→行动→观察→循环

七、实战案例

- [七、实战案例](#) — 3 个真实场景

八、多 Agent 框架与开发实战

- [八、多 Agent 框架与开发实战](#) — 框架选型 + 代码示例
 - [8.1 主流框架选型](#) — LangChain 示例
 - [8.2 AutoGen 框架](#) — 核心特性详解
 - [8.3 实战：相声](#) — 两个 Agent 讲相声
 - [8.4 代码审查](#) — 代码审查团队
 - [8.5 AutoGen 进阶：人类参与](#) — 人机协作
 - [8.6 CrewAI 框架](#) — 基于角色的多 Agent 框架
 - [8.7 CrewAI 实战：博客](#) — 写技术博客
 - [8.8 CrewAI 实战：旅行](#) — 旅行规划团队
 - [8.9 CrewAI 进阶：层级执行](#) — 层级执行流程
 - [8.10 AutoGen vs CrewAI](#) — 如何选择？
 - [8.11 手动实现](#) — 理解原理

九、挑战与局限

- [九、Agent 的挑战与局限](#) — 5 个核心问题

十、面试常见问题

- [十、面试常见问题](#) — 10 个高频面试题

总结与速查

- [总结一句话](#)
- [📖 术语对照索引](#)
- [自检清单](#)

〇、学完本章你能做什么

- ☒ 理解 Agent 是什么，和 Prompt/Skill/Workflow 有什么区别
- ☒ 知道 4 种主流开发范式（简单 LLM / 单智能体 / 工作流 / 多智能体）
- ☒ 了解 Agent 的核心组成（LLM + 工具 + 记忆 + 规划）
- ☒ 能描述 Agent 的工作流程（思考→行动→观察→循环）

一、从一个场景开始

一个你可能遇到过的场景

你让 AI 帮你订机票：

没有 Agent 时：

你：帮我订一张明天从北京到上海的机票
AI：好的，以下是订票步骤：

1. 打开携程网站
2. 选择出发城市：北京
3. 选择到达城市：上海
4. 选择日期：明天
5. 点击搜索
...（给你一堆文字说明）
你：（自己手动操作）

有 Agent 时：

你：帮我订一张明天从北京到上海的机票
AI：好的，我来帮你订。
[Agent 自动调用携程 API，查询航班]
[Agent 自动选择最优航班，填写信息]
[Agent 自动完成支付（需要你确认）]
AI：已帮你订好，航班 MU5101，明天 8:00 起飞，票价 800 元。
需要我发送行程单到你邮箱吗？

思考：两个 AI 有什么区别？

第一个 AI 只能"告诉你怎么做"，第二个 AI 能"帮你做"。第二个 AI 就是一个 Agent。

💡 一句话总结：Agent = 能自主决策 + 能调用工具 + 能完成任务的 AI。

二、Agent 是什么

2.1 通俗理解

我们可以将 Agent 🧠（智能体）理解为一个具备**自主理解、规划、记忆和工具使用能力**的数字化实体。

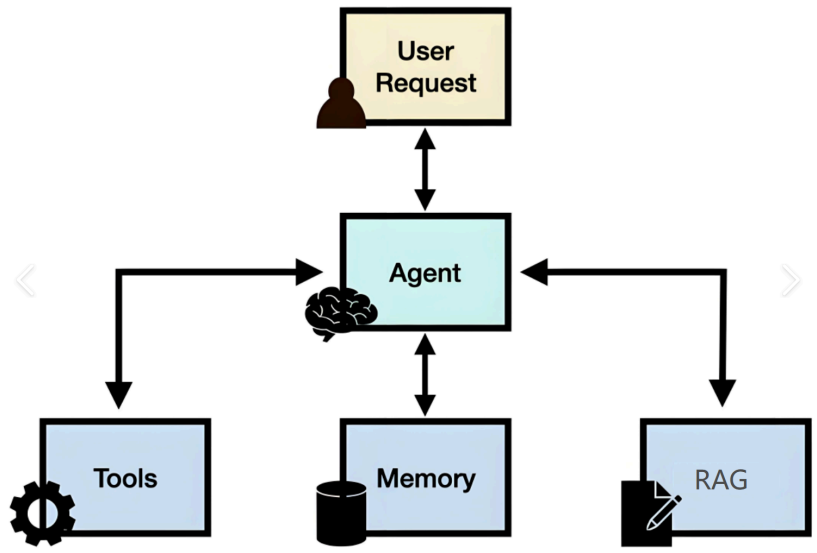
类比：

概念	类比	说明
Prompt	你口头交代任务	说一次，做一次
Skill 📖	员工手册	你定好规矩，AI 照着做
Agent 🧠	一个能干的员工	你交代目标，AI 自己判断怎么做

生活例子：

想象一个高度智能的个人助理，你只需告诉他"帮我规划一次去北京的周末旅行"，他就能自主完成：

- **感知**：理解你的自然语言指令
- **规划与推理**：把任务拆解成查询机票、筛选酒店、规划景点、预估预算
- **记忆**：记住你之前的偏好（靠窗座位、经济型酒店）
- **工具**：调用机票预订 API、酒店查询系统、地图服务
- **反馈与迭代**：把行程草案反馈给你，根据你的意见调整



2.2 核心特征：四大能力

一个 Agent 必须具备以下四个能力：

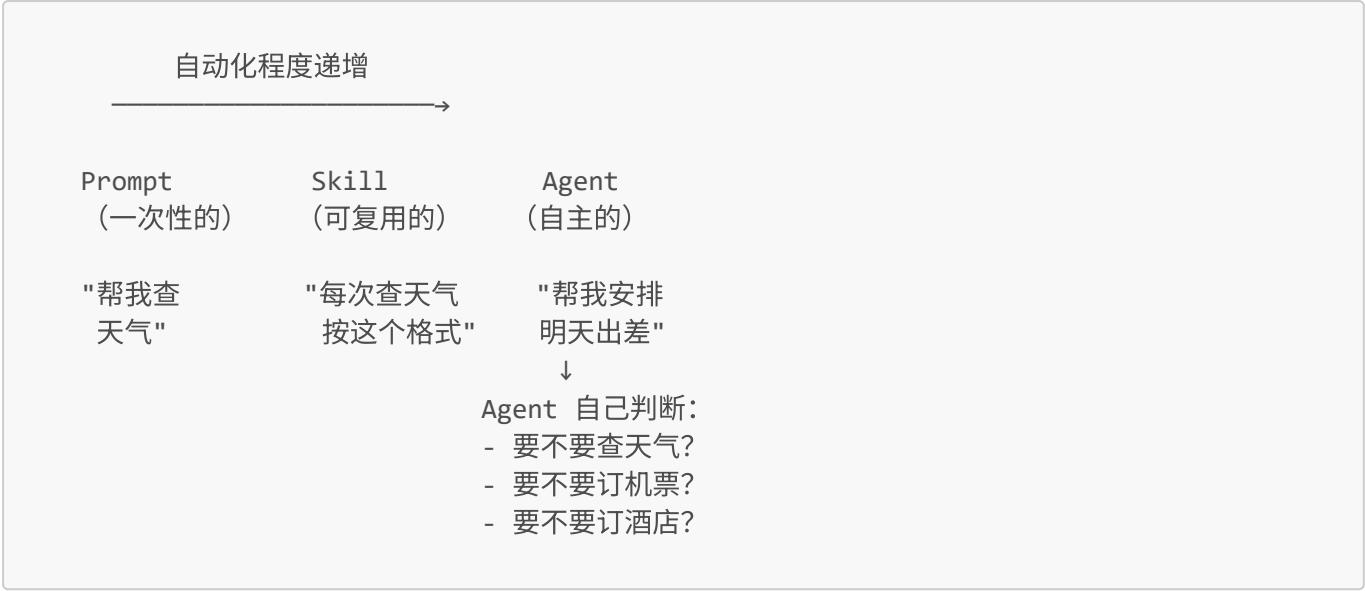
能力	说明	类比
自主决策	能自己判断下一步做什么	员工自己判断"要不要查天气"
工具调用	能调用外部工具（API、数据库、函数）	员工会用携程、钉钉、邮件
记忆能力	能记住之前的对话和上下文	员工记得你之前说过什么
规划能力	能把复杂任务拆解成多个步骤	员工会制定"出差计划"

思考：如果没有"工具调用"能力，Agent 能做什么？

只能"说"，不能"做"——就像一个只会给建议但不会动手的员工。

💡 和术语手册的联系：Function Calling 📖 = Agent 的"手脚"，让它能调用外部工具。没有 Function Calling，Agent 就是一个只会说话的"嘴"。

三、能力阶梯：从 Prompt 到 Agent



三级对比：

级别	你做什么	AI 做什么	自动化程度
Prompt	你说一次	AI 做一次	★ 最低
Skill	你写好手册	AI 按手册做	★★ 中等
Agent	你说目标	AI 自己判断怎么做	★★★ 最高

思考：什么时候用 Skill，什么时候用 Agent？

- **Skill**：流程固定、步骤明确的任务（如代码审查、写周报）
 - **Agent**：需要灵活判断、多步骤协作的任务（如安排出差、处理客服投诉）
- 💡 记住：Skill 是"你定好规矩，AI 照着做"；Agent 是"你说目标，AI 自己判断怎么做"。

四、主流开发范式

随着智能体技术的不断成熟，业界出现了多种有效的开发范式。根据解决问题的场景与复杂度，我们可以将主流的智能体应用开发范式分为 **4 种**：



4.1 简单 LLM 应用

定义：直接调用模型 API 实现内容生成的应用，完全依赖模型服务。



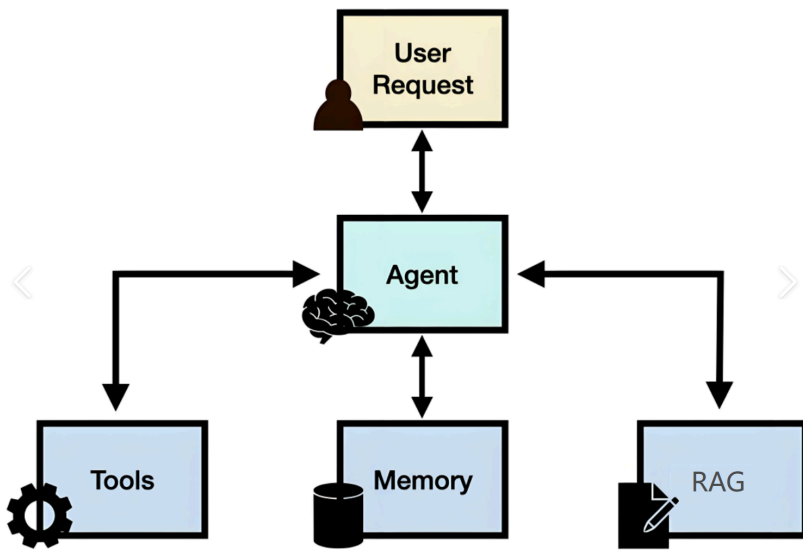
特点：

- 最简单，只是"聊天"
- 不能调用工具，不能执行动作

- 例如：ChatGPT、文心一言的早期版本

4.2 单智能体 (Single Agent)

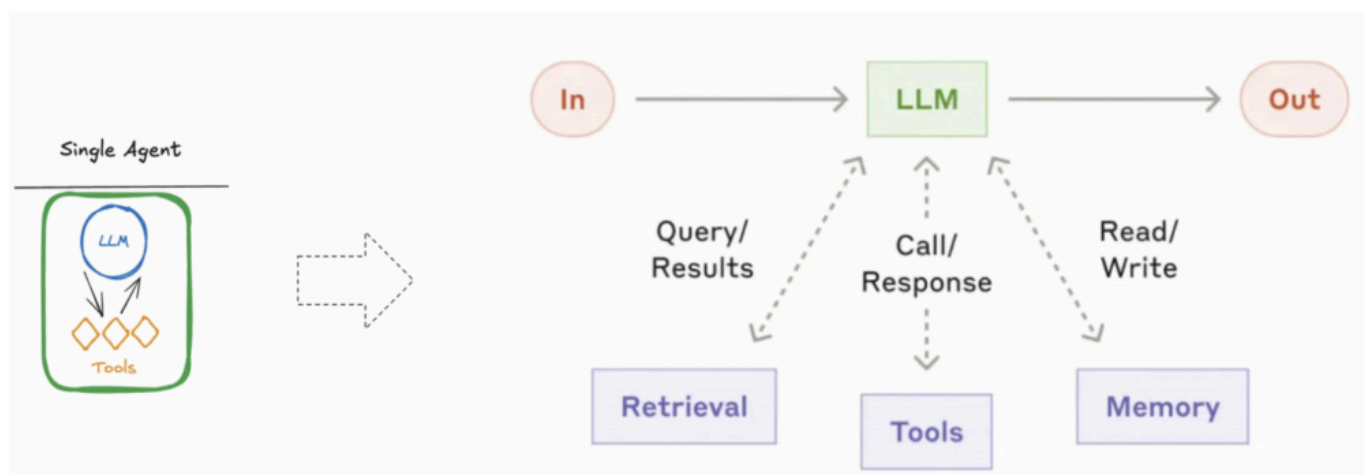
定义：为普通 LLM 应用增加 RAG、Tool、Memory 等，让模型具备与特定环境交互的能力。



特点：

- 一个 Agent 处理所有任务
- 由模型驱动，自主决策
- 例如：AI 客服、编程助手

典型架构：

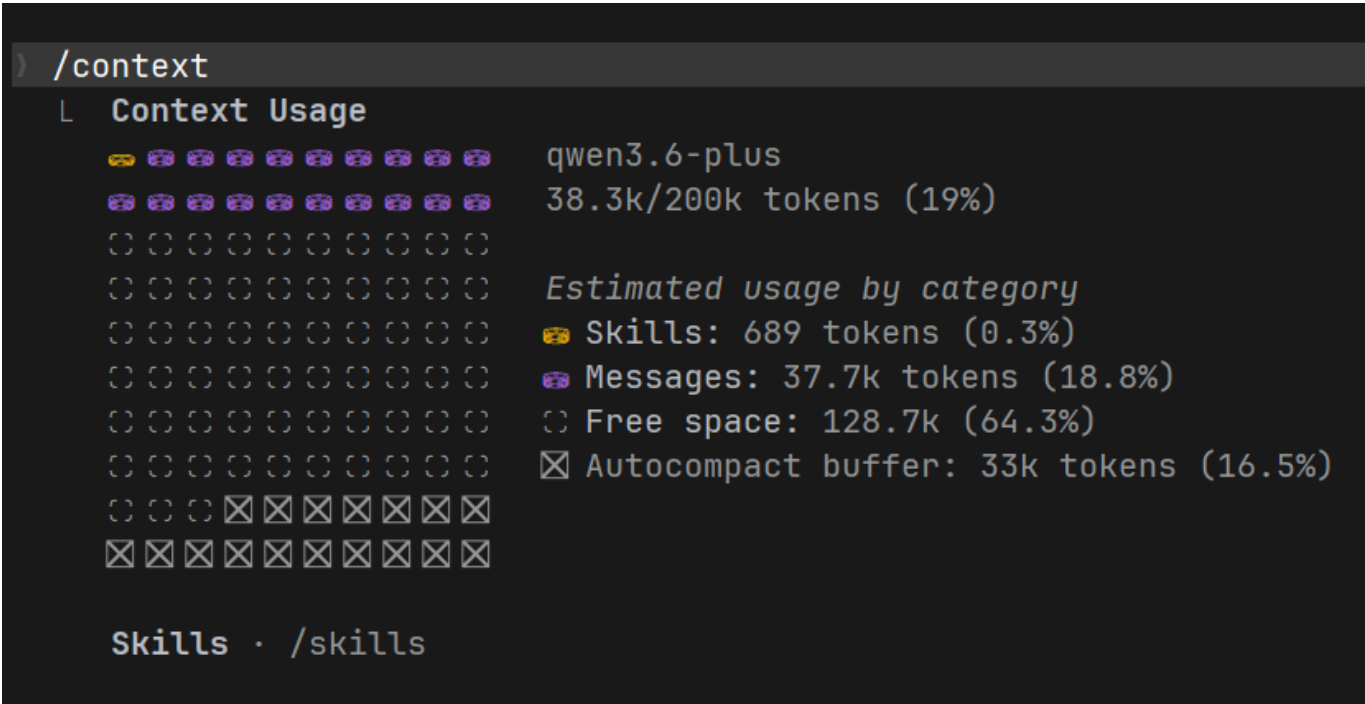


图片出自 Anthropic 博客《Building Effective AI Agents》

说它**简单**，是因为不用做智能体拆分编排，开发者只需要把工具等上下文都给到模型，一切都由模型驱动。说它**高级**，是因为整个应用由模型作为大脑来驱动和决策。

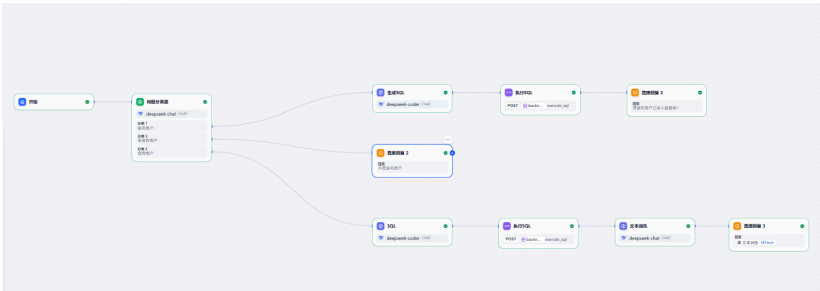
单智能体的挑战：

问题	说明
工具过多	如果 Agent 包含太多可用 Tools，模型会在决策时无所适从
上下文膨胀	多轮执行下来，消息上下文会变得很大，消耗大量 Token
复杂任务	很多任务需要多个步骤，还需要专业领域技能支持
可维护性	单 Agent 在复杂任务场景下的可维护性会变差



4.3 工作流 (Workflow)

定义：将复杂任务拆解为一系列有序步骤，步骤之间的调用关系和条件分支在设计阶段就明确。



特点：

- 确定性强、可调试性高、可控性好
- 像固定的流水线，强调任务执行的透明度和安全性
- 适合流程清晰、可拆解的任务

两种典型工作流：

① Chain - 链式工作流

将复杂任务分解为一系列较小、清晰的子任务。每一步由独立的 LLM 调用完成，上一阶段的输出作为下一阶段的输入。

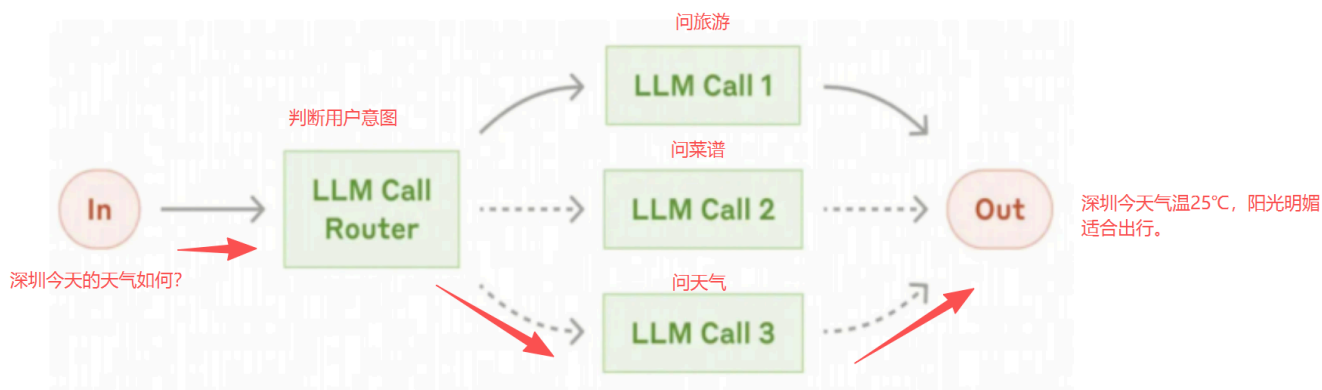


示例:

1. 生成营销文案
2. 翻译成另一种语言
3. 检查是否符合要求

② Routing - 路由 workflow

通过对输入进行分类, 将它们分派到专门设计的下游任务或处理路径 (意图识别)。

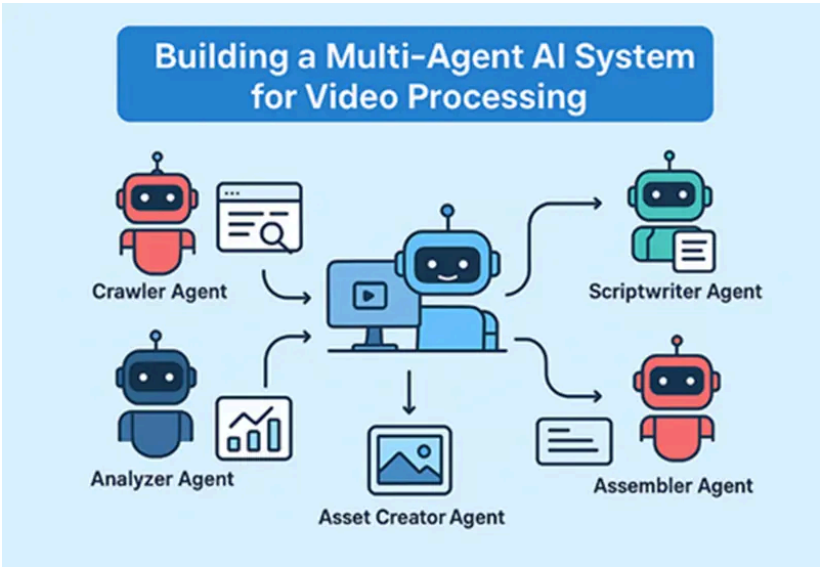


示例:

- 客服系统中, 普通咨询、退款请求、技术支持路由到不同处理流程
- 成本优化: 简单请求分发给小模型, 复杂问题交给大模型

4.4 多智能体系统 (Multi-Agent)

定义: 由多个具备自治能力的 Agent 组成, 每个 Agent 有自己的角色、能力或工具。它们通过消息或上下文交互, 协同完成复杂目标。



特点：

- 更接近团队合作，而不是预先定义好的流程
- 灵活性和适应性强
- 适合任务边界不清晰、需要动态规划的场景

对比 workflow：

维度	workflow	多智能体
流程	预定义，确定性	模型驱动，对话式
可控性	高	低
灵活性	低	高
适用场景	流程清晰	任务边界不清晰

思考：什么时候用 workflow，什么时候用多智能体？

- **workflow**：流程明确、需要稳定性的场景（如客服系统、审批流程）
- **多智能体**：需要灵活协作、动态决策的场景（如研究型问答、复杂数据分析）

💡 和术语手册的联系：**Workflow** = 固定流程编排；**Multi-Agent** = 动态协作。选择哪个取决于场景的确定性和灵活性需求。

4.4.1 多 Agent 协作：为什么需要“团队作战”？

单 Agent 的局限性：

想象一个场景：你让一个 Agent “写一篇关于 AI 的技术博客”。

单 Agent 的做法：

1. 自己查资料（调用搜索工具）

2. 自己写大纲

3. 自己写正文

- 4. 自己检查语法
- 5. 自己配图
- ...

问题：

- 一个 Agent 要做所有事，容易"分心"
- 每个环节都需要不同的"专业技能"
- 上下文越来越长，Token 消耗巨大
- 出了 bug 很难定位是哪一步的问题

多 Agent 协作的做法：

分工协作：

- 研究员 Agent：负责查资料、整理数据
- 写作 Agent：负责写文章
- 校对 Agent：负责检查语法和逻辑
- 配图 Agent：负责生成配图

优势：

- ✓ 每个 Agent 专注做好一件事（单一职责）
- ✓ 可以并行工作（研究员查资料的同时，写作 Agent 可以先写大纲）
- ✓ 容易调试（哪个环节出问题，就找哪个 Agent）
- ✓ 可以复用（校对 Agent 可以用在其他项目）

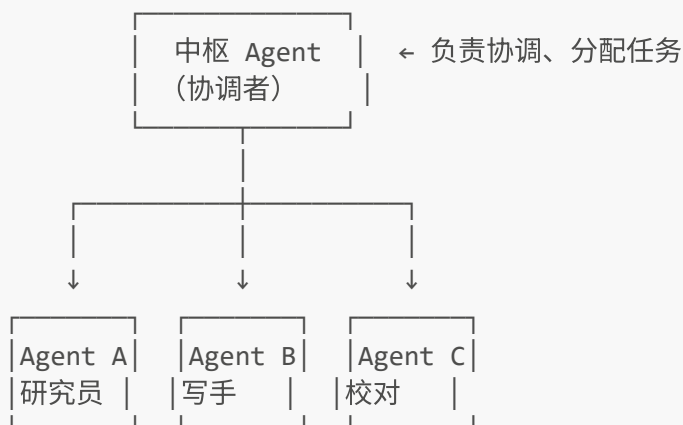
类比：

- **单 Agent** = 一个人又要写代码、又要做设计、又要写文档、又要测试——累死
- **多 Agent** = 一个团队：程序员、设计师、技术文档工程师、测试工程师——各司其职

4.4.2 多 Agent 的通信机制

多个 Agent 怎么"说话"？怎么"协作"？这需要一个**通信机制**。

模式 1：中枢 Agent 模式（Hub and Spoke）★ 最常用



工作原理：

1. 中枢 Agent 接收用户任务
2. 中枢 Agent 决定"这个任务需要哪些 Agent 参与"
3. 中枢 Agent 把子任务分配给各个 Agent
4. 各个 Agent 完成后，把结果返回给中枢 Agent
5. 中枢 Agent 汇总结果，输出最终答案

示例：

用户：写一篇关于 AI 的技术博客

中枢 Agent：

- 分配给研究员 Agent："查一下 AI 最新进展"
- 分配给写手 Agent："根据资料写文章"
- 分配给校对 Agent："检查语法和逻辑"
- 汇总结果，输出最终文章

优点：

- 结构清晰，容易理解
- 中枢 Agent 掌控全局，不容易混乱
- 容易添加新的 Agent（只需告诉中枢 Agent）

缺点：

- 中枢 Agent 是单点故障，挂了整个系统就挂了
- 中枢 Agent 负载重，所有通信都要经过它

模式 2：链式通信（Chain）

Agent A → Agent B → Agent C → 输出
(研究) (写作) (校对)

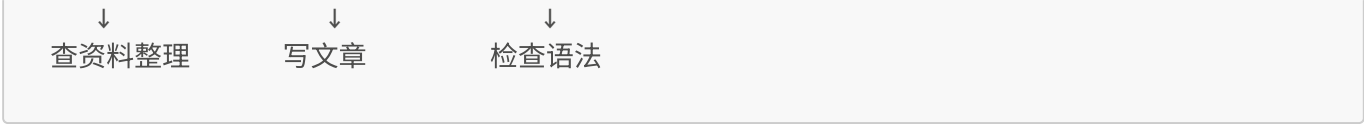
工作原理：

1. Agent A 完成任务，把结果传给 Agent B
2. Agent B 基于 Agent A 的结果继续处理，传给 Agent C
3. Agent C 最后处理，输出结果

示例：

用户：写一篇关于 AI 的技术博客

研究员 Agent → 写手 Agent → 校对 Agent → 输出



优点：

- 流程清晰，像流水线
- 每个 Agent 只关心"上一步的结果"

缺点：

- 不能并行，必须按顺序
- 如果中间某个 Agent 出错，整个链条就断了

模式 3：广播式通信（Broadcast）



工作原理：

1. 任何 Agent 都可以发布消息到共享队列
2. 所有 Agent 都能看到消息
3. 每个 Agent 自己决定"这条消息和我有没有关系"

示例：

研究员 Agent 发布："找到了 10 篇关于 AI 的论文"

所有 Agent 都收到这条消息：

- 写手 Agent："好的，我根据这些论文写文章"
- 校对 Agent："等我检查一下论文的引用格式"
- 配图 Agent："我根据论文内容生成配图"

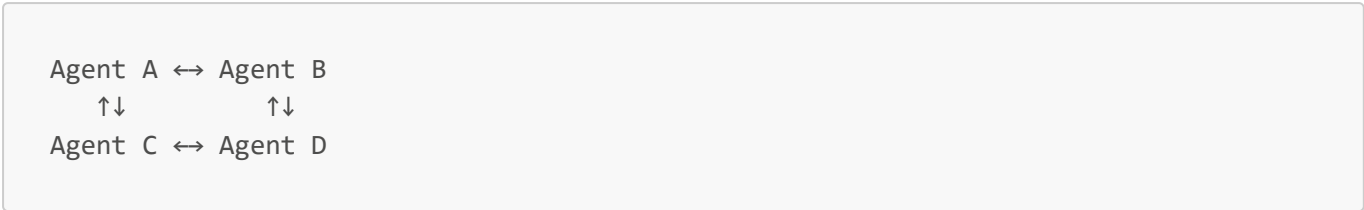
优点：

- 高度灵活，Agent 之间松耦合
- 可以并行处理

缺点：

- 消息太多，Agent 可能"看不过来"
- 难以追踪"谁在什么时候做了什么"

模式 4：点对点通信（Peer-to-Peer）



工作原理：

1. Agent 之间直接通信，不需要中枢
2. 每个 Agent 知道"我应该和谁协作"

示例：

写手 Agent 直接问研究员 Agent："帮我查一下 GPT-4 的最新进展"
研究员 Agent 直接回复："好的，这是最新的 5 篇论文..."

优点：

- 通信效率高，不经过中枢
- 适合"固定搭档"的场景

缺点：

- 结构复杂，Agent 之间关系网难以管理
- 新增 Agent 需要修改多个连接

通信模式对比

模式	适用场景	优点	缺点
中枢 Agent	任务需要协调	结构清晰，易管理	单点故障
链式	流程固定，按顺序执行	流程清晰	不能并行
广播式	需要高度灵活	松耦合，可并行	消息爆炸
点对点	固定搭档	通信效率高	结构复杂

实际业务场景详解

下面通过 8 个真实业务场景，详细说明每种通信机制适合什么场景。

场景 1：AI 客服系统（中枢 Agent 模式）★

业务场景：

```
用户: "我要退货"
↓
[客服中枢 Agent]
↓
├─ 订单查询 Agent: "查订单信息"
├─ 退货政策 Agent: "判断是否符合退货条件"
├─ 退货处理 Agent: "生成退货单"
└─ 物流跟踪 Agent: "安排取件"
↓
[汇总结果, 回复用户]
```

为什么用中枢 Agent?

- ☒ 用户请求需要**协调多个系统**（订单、退货政策、物流）
- ☒ 需要**统一入口**（所有请求都从客服入口进）
- ☒ 需要**汇总结果**（把多个系统的信息整合后回复用户）
- ☒ 流程**相对固定**（退货流程是标准化的）

实际案例:

```
# 伪代码示例
class CustomerServiceHub:
    def handle_return_request(self, user_message):
        # 1. 查询订单
        order_info = self.order_agent.query_order(user_message)

        # 2. 检查退货政策
        policy_check = self.policy_agent.check_return_policy(order_info)

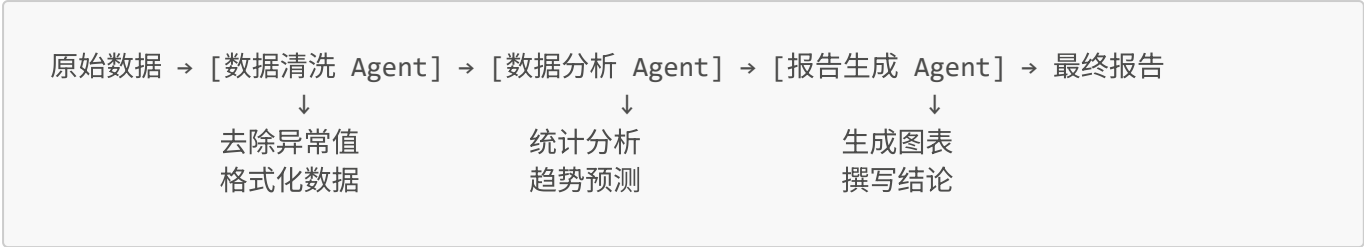
        # 3. 处理退货
        if policy_check.can_return:
            return_form = self.return_agent.create_return_form(order_info)
            logistics = self.logistics_agent.arrange_pickup(return_form)
            return f"已为您生成退货单 {return_form.id}, 快递员将在 {logistics.time}
上门取件"
        else:
            return f"抱歉, 您的订单不符合退货条件: {policy_check.reason}"
```

类似场景:

- 银行客服（查余额、转账、理财咨询）
- 电商客服（查物流、退换货、投诉处理）
- 医院挂号系统（挂号、问诊、缴费）

场景 2: 数据处理流水线（链式通信）★

业务场景:



为什么用链式通信？

- ☒ 流程**严格顺序**（必须先清洗，再分析，再生成报告）
- ☒ 每个 Agent **只关心上一步的输出**
- ☒ 流程像**流水线**，一环扣一环
- ☒ **不需要并行**（数据分析必须等数据清洗完成）

实际案例：

```
# 伪代码示例
class DataPipeline:
    def process_sales_data(self, raw_data):
        # Step 1: 数据清洗
        clean_data = self.cleaner_agent.clean(raw_data)
        # 去除异常值、填充缺失值、格式化日期

        # Step 2: 数据分析（必须等清洗完成）
        analysis = self.analyzer_agent.analyze(clean_data)
        # 计算销售额、增长率、趋势预测

        # Step 3: 报告生成（必须等分析完成）
        report = self.reporter_agent.generate_report(analysis)
        # 生成图表、撰写结论、输出 PDF

        return report
```

类似场景：

- ETL 数据处理（Extract → Transform → Load）
- 文档处理（OCR → 文本提取 → 内容分析）
- 视频处理（解码 → 帧提取 → 目标检测 → 编码）

场景 3：实时监控告警系统（广播式通信）★

业务场景：



```
└─> 运维 Agent: "检查服务器状态"
└─> 数据库 Agent: "检查数据库连接"
└─> 业务 Agent: "检查业务影响"
└─> 通知 Agent: "发送告警给相关人员"
```

为什么用广播式通信？

- ☒ 事件需要多个团队同时响应
- ☒ 每个 Agent 自己判断"这个事件和我有没有关系"
- ☒ 松耦合（增加新的响应 Agent 不需要修改其他 Agent）
- ☒ 并行处理（多个 Agent 可以同时响应）

实际案例：

```
# 伪代码示例
class MonitoringSystem:
    def on_alert(self, alert):
        # 发布告警到共享队列
        self.message_bus.publish("alert", alert)

        # 所有订阅的 Agent 都会收到
        # 运维 Agent 订阅了 "alert"
        # 数据库 Agent 订阅了 "alert"
        # 业务 Agent 订阅了 "alert"
        # 通知 Agent 订阅了 "alert"

        # 每个 Agent 自己决定是否处理
        # 例如：数据库 Agent 只在 alert.type == "database" 时处理
```

类似场景：

- 股票交易系统（价格波动触发多个策略）
- 物联网系统（传感器数据触发多个处理逻辑）
- 社交媒体（新帖子触发内容审核、推荐、通知）

场景 4：代码审查配对（点对点通信）★

业务场景：

```
[代码阅读 Agent] ↔ [Bug 检测 Agent]
    ↑↓                ↑↓
[测试生成 Agent] ↔ [文档更新 Agent]
```

为什么用点对点通信？

- ☒ Agent 之间是固定搭档（代码阅读和 Bug 检测总是配对）

- ☒ 需要频繁双向沟通 ("这个函数有问题" "请详细说明")
- ☒ 不需要中枢 (两个 Agent 直接对话即可)
- ☒ 通信效率高 (不经过中枢, 直接对话)

实际案例:

```
# 伪代码示例
class CodeReviewPair:
    def review_code(self, code):
        # 代码阅读 Agent 和 Bug 检测 Agent 直接对话
        analysis = self.code_reader.read(code)

        # Bug 检测 Agent 直接问代码阅读 Agent
        bugs = self.bug_finder.find_bugs(analysis)

        # 代码阅读 Agent 补充信息
        for bug in bugs:
            context = self.code_reader.get_context(bug.location)
            bug.add_context(context)

        return bugs
```

类似场景:

- 翻译配对 (中文翻译 ↔ 英文翻译)
- 设计配对 (UI 设计师 ↔ 交互设计师)
- 测试配对 (测试用例编写 ↔ 测试执行)

场景 5: 智能写作助手 (混合模式: 中枢 + 链式) ★

业务场景:

```
用户: 写一篇技术博客
↓
[写作中枢 Agent]
↓
├→ [大纲生成 Agent] (链式第 1 步)
│   ↓
├→ [内容填充 Agent] (链式第 2 步)
│   ↓
├→ [语法检查 Agent] (链式第 3 步)
│   ↓
└→ [最终输出]
```

为什么用混合模式?

- ☒ 全局用中枢 (写作中枢协调整个流程)

- ☒ **内部用链式**（大纲 → 内容 → 语法检查是顺序流程）
- ☒ **灵活性 + 可控性**（中枢可以动态调整，链式保证流程）

实际案例：

```
# 伪代码示例
class WritingAssistant:
    def write_article(self, topic):
        # 中枢：决定写作流程
        outline = self.outline_agent.generate(topic)

        # 链式：顺序执行
        content = self.content_agent.fill(outline)
        checked = self.grammar_agent.check(content)

        return checked
```

类似场景：

- 论文写作助手（选题 → 大纲 → 正文 → 校对）
- 营销文案生成（调研 → 创意 → 撰写 → 审核）
- 简历生成（信息收集 → 排版 → 优化 → 输出）

场景 6：多模态内容创作（广播式 + 中枢）★

业务场景：

```
用户：创建一个产品介绍视频
↓
[创作中枢 Agent]
↓
发布任务到共享队列
↓
所有 Agent 都收到：
├→ 文案 Agent："写视频脚本"
├→ 配音 Agent："生成语音"（等文案完成）
├→ 配图 Agent："生成配图"（独立并行）
├→ 视频 Agent："合成视频"（等所有素材完成）
└→ 字幕 Agent："生成字幕"（等配音完成）
```

为什么用混合模式？

- ☒ **广播式**：多个 Agent 同时响应任务
- ☒ **中枢**：视频 Agent 需要等待所有素材完成
- ☒ **并行 + 串行**：配图可以并行，配音必须等文案

实际案例：

```
# 伪代码示例
class VideoCreation:
    def create_video(self, product_info):
        # 广播：所有 Agent 同时开始
        self.message_bus.publish("task", product_info)

        # 并行：文案和配图同时进行
        script = self.copywriter_agent.write(product_info)
        images = self.image_agent.generate(product_info)

        # 串行：配音必须等文案
        audio = self.voice_agent.generate(script)

        # 串行：字幕必须等配音
        subtitles = self.subtitle_agent.generate(audio)

        # 串行：视频合成必须等所有素材
        video = self.video_agent.compose(script, audio, images, subtitles)

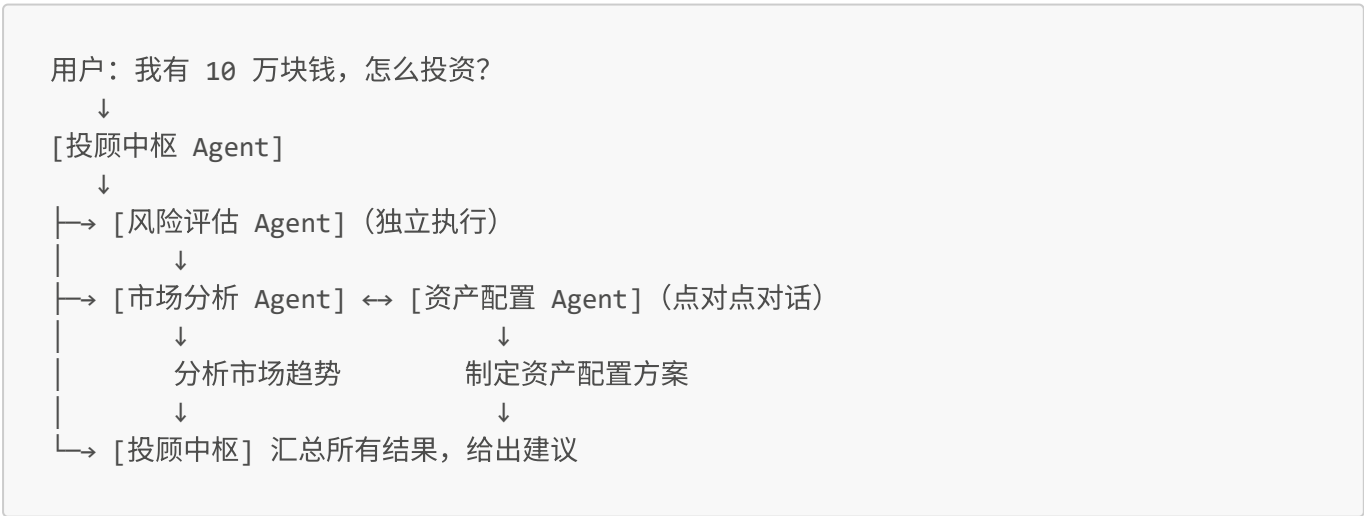
        return video
```

类似场景：

- 游戏开发（策划 → 美术 → 程序 → 测试）
 - 电影制作（编剧 → 拍摄 → 剪辑 → 特效）
 - 音乐制作（作词 → 作曲 → 编曲 → 录音 → 混音）
-

场景 7：智能投顾系统（中枢 + 点对点） ★

业务场景：



为什么用混合模式？

-  **中枢：**投顾中枢协调所有分析

- ☒ **点对点**：市场分析和资产配置需要频繁对话（"市场趋势如何？" "基于这个趋势，建议配置 60% 股票"）

实际案例：

```
# 伪代码示例
class InvestmentAdvisor:
    def advise(self, user_info):
        # 中枢：协调所有分析
        risk = self.risk_agent.assess(user_info)

        # 点对点：市场分析和资产配置对话
        market_analysis = self.market_agent.analyze()
        allocation = self.allocation_agent.plan(market_analysis, risk)

        # 多次对话
        for i in range(3): # 迭代优化
            feedback = self.market_agent.feedback(allocation)
            allocation = self.allocation_agent.adjust(feedback)

        # 中枢：汇总结果
        return self.hub.summarize(risk, market_analysis, allocation)
```

类似场景：

- 医疗诊断（全科医生协调，专科医生会诊）
- 法律咨询（主办律师协调，专业律师提供意见）
- 教育辅导（班主任协调，各科老师辅导）

场景 8：分布式任务队列（广播式）★

业务场景：

```
[任务调度器] 发布任务："处理 1000 张图片"
↓
共享消息队列
↓
100 个 Worker Agent 都收到任务
↓
每个 Worker 自己领取任务：
├→ Worker 1：处理图片 1-10
├→ Worker 2：处理图片 11-20
├→ Worker 3：处理图片 21-30
...
└→ Worker 100：处理图片 991-1000
```

为什么用广播式？

- ☒ **高度并行**（100 个 Worker 同时处理）
- ☒ **动态扩展**（增加 Worker 不需要修改其他组件）
- ☒ **负载均衡**（每个 Worker 自己领取任务）
- ☒ **松耦合**（Worker 之间不需要通信）

实际案例：

```
# 伪代码示例（类似 Celery、RabbitMQ）
class TaskQueue:
    def process_images(self, image_urls):
        # 发布任务到队列
        for url in image_urls:
            self.queue.publish("process_image", url)

        # 100 个 Worker Agent 自动领取任务
        # 每个 Worker 处理一张图片
        # 处理完成后发布结果到结果队列

        # 收集所有结果
        results = self.result_queue.collect(len(image_urls))
        return results
```

类似场景：

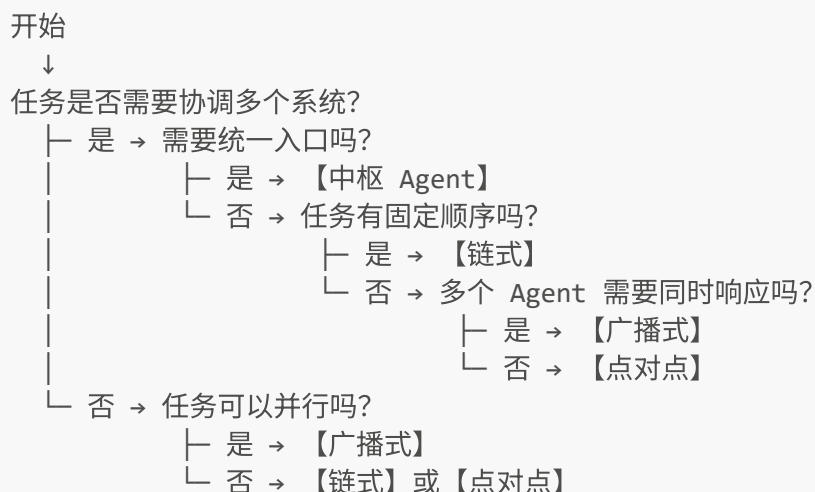
- 爬虫系统（分发 URL，多个爬虫并行抓取）
- 数据处理（分发数据块，多个 Worker 并行处理）
- 渲染农场（分发渲染任务，多个节点并行渲染）

通信机制选择指南

快速决策表：

你的场景	推荐通信机制	原因
需要统一入口，协调多个系统	中枢 Agent	结构清晰，易管理
流程固定，按顺序执行	链式	流程清晰，像流水线
多个 Agent 需要同时响应事件	广播式	并行处理，松耦合
固定搭档，频繁双向沟通	点对点	通信效率高
既有全局协调，又有顺序流程	中枢 + 链式	灵活 + 可控
既有并行处理，又有依赖关系	广播 + 中枢	并行 + 串行
大规模并行任务	广播式	高度并行，动态扩展

决策流程图：



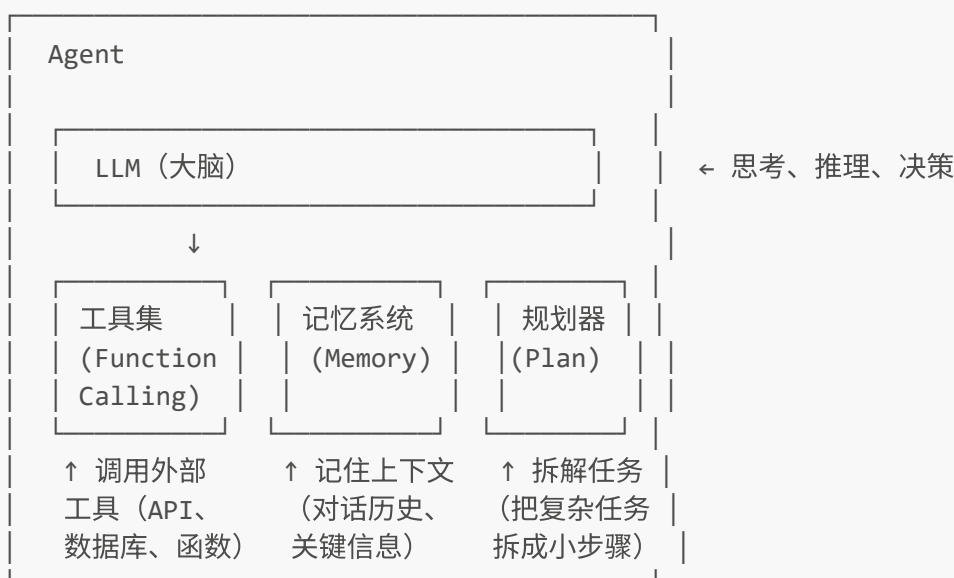
实际项目建议：

1. **从简单开始**：先用单 Agent，不够用再加多 Agent
2. **优先中枢**：大多数场景用中枢 Agent 就够了
3. **按需混合**：复杂场景再考虑混合模式
4. **监控调试**：多 Agent 系统一定要加日志，方便调试

💡 **记住**：没有“最好”的通信机制，只有“最合适”的。根据业务场景选择，而不是为了用技术而用技术。

五、Agent 的组成部分

一个完整的 Agent 由以下四部分组成：



逐部分讲解

① LLM（大脑）

- 负责思考、推理、决策

- 决定"下一步做什么"
- 常用：GPT-4、Claude、Qwen

② 工具集 (Function Calling 📖)

- 让 Agent 能调用外部工具
- 例如：查天气 API、订机票 API、查数据库、发邮件

③ 记忆系统 (Memory)

- 短期记忆：当前对话的上下文（用 Context Engineering 📖 管理）
- 长期记忆：存储到向量数据库，跨对话记忆

④ 规划器 (Planning)

- 把复杂任务拆解成多个步骤
- 例如："安排出差" → 查天气 → 订机票 → 订酒店 → 安排接送

思考：如果 Agent 没有"记忆系统"，会有什么问题？

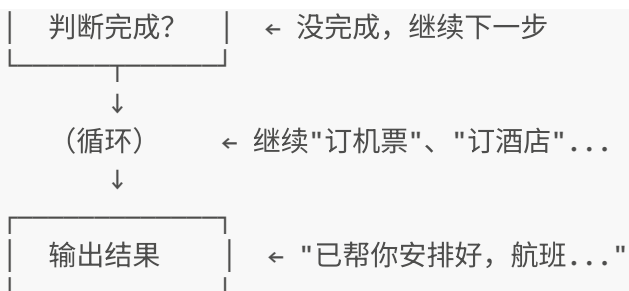
它会"失忆"——每次对话都从头开始，不记得你之前说过什么。就像一个新员工，每次都问"您之前说的那个项目是什么？"

💡 和术语手册的联系：Context Engineering 📖 = 管理 Agent 的"短期记忆"；向量数据库 📖 = 管理 Agent 的"长期记忆"。

六、Agent 是怎么工作的？

Agent 的工作流程是一个**循环**：





一个完整的例子

用户输入：帮我安排明天去北京出差

Agent 工作流程：

[循环 1]

思考：用户要安排出差，我需要先查天气

行动：调用天气 API，查询明天北京天气

观察：明天北京晴，气温 15-25°C

判断：没完成，继续

[循环 2]

思考：天气不错，需要订机票

行动：调用携程 API，查询北京→上海航班

观察：最优航班 MU5101，8:00 起飞，800 元

判断：没完成，继续

[循环 3]

思考：需要订酒店

行动：调用酒店预订 API，查询北京酒店

观察：如家酒店，300 元/晚

判断：没完成，继续

[循环 4]

思考：需要安排接送机

行动：调用打车 API，预约明天 6:00 接送

观察：已预约成功

判断：完成，输出结果

[输出]

已帮你安排好明天去北京出差：

- 天气：晴，15-25°C
- 航班：MU5101，8:00 起飞，800 元
- 酒店：如家酒店，300 元/晚
- 接送：已预约明天 6:00 接送机

记住：Agent 的工作是“循环”的——思考→行动→观察→再思考→再行动→...直到任务完成。

七、实战案例

案例 1：AI 客服 Agent

场景：用户咨询退货

Agent 工作流程：

用户：我要退货

Agent 思考：用户要退货，我需要先了解订单信息
行动：调用订单查询 API
观察：用户订单 #12345，购买 iPhone 16，7 天前

Agent 思考：符合退货条件（7 天内），需要收集退货原因
行动：回复用户"请问退货原因是什么？"

用户：屏幕有坏点

Agent 思考：原因是质量问题，需要生成退货单
行动：调用退货 API，生成退货单
观察：退货单 #RMA-789

Agent 思考：需要告知用户退货流程
行动：回复用户"已生成退货单 #RMA-789，请将商品寄回..."

案例 2：数据分析 Agent

场景：用户问"上个月销售额最高的产品是什么？"

Agent 工作流程：

Agent 思考：需要查询数据库
行动：生成 SQL 查询语句
SELECT product_name, SUM(sales) as total
FROM orders
WHERE created_at >= '2024-01-01'
AND created_at < '2024-02-01'
GROUP BY product_name
ORDER BY total DESC
LIMIT 1
观察：查询结果：iPhone 16，销售额 120 万

Agent 思考：需要生成可视化图表
行动：调用 Python 生成柱状图
观察：图表生成成功

Agent 思考：需要给用户总结
行动：回复用户"上个月销售额最高的是 iPhone 16，
销售额 120 万元。以下是详细图表：" [附上柱状图]

案例 3：编程助手 Agent

场景：用户说"这段代码有 bug，帮我调试"

Agent 工作流程：

```
Agent 思考：需要先读取代码
行动：读取 test_code.py
观察：代码内容...

Agent 思考：需要运行测试，看哪里报错
行动：执行 pytest test_code.py
观察：测试失败，错误信息：TypeError at line 42

Agent 思考：需要分析错误原因
行动：查看 line 42，发现类型不匹配
观察：price 是字符串，应该转成浮点数

Agent 思考：需要修复 bug
行动：修改代码，添加类型转换
观察：修复完成

Agent 思考：需要再次测试，确认修复
行动：执行 pytest test_code.py
观察：测试通过

Agent 思考：任务完成
行动：回复用户"已修复，问题是 line 42 类型不匹配，
      已添加类型转换。测试已通过。"
```

八、多 Agent 框架与开发实战

 本章聚焦**多 Agent 框架**（AutoGen、CrewAI）和**开发方法**（用框架 vs 手动实现）。

8.1 主流框架选型与 LangChain 示例

主流框架对比：

框架	特点	适用场景
LangChain 	最流行，组件丰富	通用 Agent 开发
LangGraph 	基于图结构，流程可控	复杂流程编排
AutoGen	多 Agent 对话，人机协作	灵活对话、需要人类参与
CrewAI	角色扮演，任务驱动	明确分工、任务依赖

LangChain 代码示例：

```

import os
import pprint

from langchain.agents import create_agent
from langchain.tools import tool
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv
load_dotenv()

# 定义工具（使用 @tool 装饰器）
@tool
def search_weather(city: str) -> str:
    """查询天气"""
    return f"{city}明天晴, 15-25°C"

@tool
def book_flight(origin: str, dest: str, date: str) -> str:
    """订机票"""
    return f"已订票: {origin}→{dest}, {date}"

tools = [search_weather, book_flight]

# 初始化模型
# 将环境变量作为参数传入（如果模型提供商兼容 OpenAI API）
model = ChatOpenAI(
    model="qwen-plus", # 根据您的服务端修改模型名称
    base_url="https://llm-p5jtn7y87smzjpwo.cn-beijing.maas.aliyuncs.com/compatible-mode/v1",
    api_key=os.getenv("DASHSCOPE_API_KEY")
)

# 初始化 Agent
agent = create_agent(model=model, tools=tools)

# 运行 Agent
response = agent.invoke({"messages": [{"role": "user", "content": "帮我安排明天去北京出差"}]})
# 将 response 转换为字典后，格式化打印
pprint.pprint(response, width=120, sort_dicts=False)

```

💡 LangChain 适合单 **Agent** 场景。如果需要多 **Agent** 协作，推荐用 AutoGen 或 CrewAI（见下文）。

8.2 AutoGen：微软的多 Agent 框架

AutoGen 是微软开源的多 Agent 对话框架，特点是**简单、灵活、好玩**。

核心思想：

把每个 Agent 想象成一个“聊天参与者”，它们通过“对话”来协作完成任务。

AutoGen 的核心特性

特性 1：对话驱动（Conversation-Driven）

AutoGen 的核心是"对话"——Agent 之间通过对话协作，就像人在聊天一样。

```
Agent A: "我找到了 3 篇相关论文"
Agent B: "好的，我根据这些论文写文章"
Agent C: "文章写好了，请检查"
Agent A: "没问题，可以发布"
```

对比 CrewAI：

- **AutoGen**：Agent 之间"聊天"，灵活但可能跑偏
- **CrewAI**：Agent 按"任务"执行，结构化但不够灵活

特性 2：多种 Agent 类型

AutoGen 提供多种内置 Agent 类型：

Agent 类型	说明	适用场景
AssistantAgent	通用助手，能调用工具	大多数场景
UserProxyAgent	代表用户，可以执行代码	需要人类参与
GroupChatManager	群聊管理器	多 Agent 对话
Custom Agent	自定义 Agent	特殊需求

示例：

```
from autogen import AssistantAgent, UserProxyAgent

# 助手 Agent (AI)
assistant = AssistantAgent(
    name="AI助手",
    system_message="你是一个有帮助的助手"
)

# 用户代理 (代表人类)
user_proxy = UserProxyAgent(
    name="用户",
    human_input_mode="ALWAYS", # 总是需要人类输入
    code_execution_config={"work_dir": "coding"}
)
```

特性 3：灵活的对话模式

AutoGen 支持多种对话模式：

模式	说明	代码示例
两人对话	两个 Agent 对话	<code>initiate_chat()</code>
群聊（轮转）	多个 Agent 轮流发言	<code>RoundRobinGroupChat</code>
群聊（选择）	动态选择下一个发言者	<code>SelectorGroupChat</code>
嵌套对话	对话中可以启动子对话	复杂场景

特性 4：人机协作（Human-in-the-Loop）

AutoGen 最重要的特性之一——让人类参与对话。

3 种人类参与模式：

模式	说明	适用场景
ALWAYS	每次都问人类	需要人类审核每一步
TERMINATE	只在结束时间问人类	只需要人类确认最终结果
NEVER	不问人类	全自动执行

示例：

```
user_proxy = UserProxyAgent(
    name="用户",
    human_input_mode="TERMINATE", # 只在结束时间问
    code_execution_config={"work_dir": "coding"}
)
```

特性 5：代码执行能力

AutoGen 的 Agent 可以执行代码（Python、Shell 等）。

示例：

```
user_proxy = UserProxyAgent(
    name="用户",
    code_execution_config={
        "work_dir": "coding", # 代码执行目录
        "use_docker": False,  # 是否用 Docker
    }
)

# Agent 可以写代码并执行
# 例如：写一个 Python 脚本，计算斐波那契数列
```

⚠ 安全提示：

- 代码执行有安全风险
- 建议在 Docker 容器中执行
- 不要执行不受信任的代码

特性 6：工具调用（Function Calling）

AutoGen 支持让 Agent 调用自定义函数。

示例：

```
import json

# 定义工具函数
def get_weather(city: str) -> str:
    """获取天气信息"""
    return f"{city}明天晴，15-25°C"

# 注册工具
assistant = AssistantAgent(
    name="助手",
    system_message="你可以调用 get_weather 函数查询天气"
)

# Agent 会自动决定何时调用工具
# 用户："北京明天天气怎么样？"
# Agent 调用 get_weather("北京")
# Agent 回复："北京明天晴，15-25°C"
```

特性 7：多种终止条件

AutoGen 支持多种对话终止条件：

条件	说明	代码示例
MaxMessageTermination	达到最大消息数	MaxMessageTermination(max_messages=10)
TextMentionTermination	提到特定文本	TextMentionTermination("完成")
TokenLimitTermination	达到 Token 限制	TokenLimitTermination(max_tokens=1000)
组合条件	多个条件组合	用 AndTermination / OrTermination

示例：

```
from autogen_agentchat.conditions import MaxMessageTermination,
TextMentionTermination

# 达到 20 条消息，或者提到"完成"时终止
termination = MaxMessageTermination(max_messages=20) | TextMentionTermination("完成")
```

AutoGen 核心特性总结

特性	说明	类比
对话驱动	Agent 之间通过对话协作	人在聊天
多种 Agent 类型	Assistant / UserProxy / 自定义	不同角色
灵活对话模式	两人 / 群聊 / 嵌套	不同场景
人机协作	人类可以参与对话	有人审核
代码执行	Agent 可以写代码并执行	会写程序
工具调用	Agent 可以调用自定义函数	会用工具
多种终止条件	灵活控制对话何时结束	知道何时停

AutoGen vs CrewAI 对比：

维度	AutoGen	CrewAI
核心概念	对话	角色 + 任务
协作方式	Agent 之间对话	按流程执行任务
人机协作	✅ 原生支持	❌ 不支持
代码执行	✅ 原生支持	❌ 不支持
工具调用	✅ 支持	✅ 支持
学习曲线	较陡	较平缓

适用场景	需要灵活对话、人机协作	需要明确分工、任务依赖
------	-------------	-------------

安装:

```
pip install pyautogen
```

8.3 实战：两个 Agent 讲相声 🤖🤖

场景：让两个 Agent 讲一段相声，一个逗哏，一个捧哏。

完整代码：

```
"""
两个 Agent 讲相声
安装依赖: pip install pyautogen
"""

import asyncio
import io
import os
import sys

from dotenv import load_dotenv

load_dotenv()

sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8')

from autogen_agentchat.agents import AssistantAgent
from autogen_agentchat.ui import Console
from autogen_ext.models.openai import OpenAIChatCompletionClient

# — 配置 LLM (用通义千问) —————
model_client = OpenAIChatCompletionClient(
    model="qwen-plus",
    api_key=os.getenv("DASHSCOPE_API_KEY"),
    base_url="https://llm-p5jtn7y87smzjpwo.cn-
beijing.maas.aliyuncs.com/compatible-mode/v1",
    model_info={
        "vision": False,
        "function_calling": True,
        "json_output": True,
        "structured_output": True,
        "family": "unknown",
    },
)

# — 创建逗哏 Agent —————
douggen = AssistantAgent(
    name="FunnyMan",
    model_client=model_client,
    system_message="""你是一个相声演员，角色是逗哏。"""
```

你的任务：

- 说笑话、抖包袱
- 每次发言不超过 50 字
- 语气幽默、接地气
- 说完后等捧哏接话""",

)

— 创建捧哏 Agent —

```
penggen = AssistantAgent(
    name="StraightMan",
    model_client=model_client,
    system_message=""你是一个相声演员，角色是捧哏。
```

你的任务：

- 接话、捧哏
- 每次发言不超过 20 字
- 常用词：嗯、啊、哦、是吗、好嘛、去你的吧
- 要配合逗哏的节奏""",

)

```
async def main():
    from autogen_agentchat.teams import RoundRobinGroupChat
    from autogen_agentchat.conditions import MaxMessageTermination

    # 创建轮转群聊（逗哏和捧哏交替发言）
    team = RoundRobinGroupChat(
        [douggen, penggen],
        termination_condition=MaxMessageTermination(max_messages=20),
    )

    print("🎤 相声开始! \n")
    await Console(team.run_stream(task="今天给大家说段相声，题目是《AI 程序员》"))

asyncio.run(main())
```

运行效果（示例）：

🎤 相声开始！

逗哏：今天给大家说段相声，题目是《AI 程序员》

捧哏：哦？ AI 程序员？

逗哏：对啊，现在 AI 都能写代码了，程序员要失业了！

捧哏：是吗？

逗哏：可不是嘛！ AI 一天能写 1000 行代码，还不吃饭不睡觉！

捧哏：好嘛，这么厉害？

逗哏：那当然！但是有个问题...

捧哏：什么问题？

逗哏：AI 写的代码，Bug 也特别多！

捧哏：去你的吧！

逗哏：程序员一看，这不就是我平时写的吗？

捧哏：嗯？
逗哏：原来 AI 是跟程序员学的！
捧哏：好嘛！
...

代码解析：

1. **创建 Agent**：AssistantAgent 创建两个 Agent，分别设置 `system_message` 定义角色
2. **创建群聊**：GroupChat 把两个 Agent 放到一个群里
3. **创建管理器**：GroupChatManager 管理对话流程
4. **开始对话**：initiate_chat 启动对话，Agent 轮流发言

💡 **记住**：AutoGen 的核心是“对话”——Agent 之间通过对话协作，就像人在聊天一样。

8.4 多 Agent 实战：代码审查团队

场景：3 个 Agent 组成代码审查团队

```

"""
=====
多 Agent 协作代码审查 -- 基于 AutoGen 0.7.x (通义千问 qwen-plus)
=====

1. 什么是 Multi-Agent (多智能体) ?
   - 单个 Agent = 一个 LLM + 一个角色设定 (system message)
   - 多 Agent = 多个不同角色的 Agent 协作完成一个复杂任务
   - 类比：一个公司里有不同岗位的职员，各司其职、相互配合

2. AutoGen 框架的核心组件：
   - AssistantAgent      : Agent 个体，拥有自己的角色设定和模型
   - RoundRobinGroupChat: 轮转群聊，Agent 按顺序依次发言
   - TextMessage         : Agent 之间传递的消息
   - Console             : 把对话过程打印到终端的辅助工具

3. 多 Agent 协作的工作流程 (以代码审查为例)：
   +-----+      +-----+      +-----+
   | 代码阅读者 |---->|  Bug  猎人  |---->| 审查总结者  |
   | (读懂代码) |    | (找问题)  |    | (汇总结论)  |
   +-----+      +-----+      +-----+
   第一步：理解代码      第二步：发现缺陷      第三步：给出建议

4. 为什么不用单个 Agent 一次性完成？
   - 单一 Agent 容易顾此失彼，遗漏某些视角
   - 拆分角色后，每个 Agent 的 system_message 更聚焦，输出质量更高
   - 后一个 Agent 能看到前一个 Agent 的完整输出，形成接力

=====
"""

```

```

import os
import sys
import io
import asyncio
from dotenv import load_dotenv

# -----
# 【知识点 1】Windows 控制台编码
# -----
# Windows 命令行默认使用 GBK 编码, 无法显示 emoji 等 Unicode 字符。
# 把 stdout 包装成 UTF-8, 这样 print() 就不会报编码错误。
# -----
sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8')

load_dotenv()

from autogen_agentchat.agents import AssistantAgent
from autogen_agentchat.conditions import MaxMessageTermination
from autogen_agentchat.ui import Console
from autogen_agentchat.teams import RoundRobinGroupChat
from autogen_ext.models.openai import OpenAIChatCompletionClient

# -----
# 【知识点 2】模型客户端 (Model Client)
# -----
# AutoGen 0.7.x 把用什么 LLM 独立出来了, 叫 model_client。
# 这样你可以轻松切换模型 (GPT-4 / Claude / qwen / 本地模型等),
# 而不需要修改每个 Agent 的创建代码。
#
# 这里的配置等价于:
#   - 模型名称    : qwen-plus (通义千问)
#   - API 地址    : DashScope 兼容 OpenAI 协议的 endpoint
#   - 认证方式    : 从环境变量 DASHSCOPE_API_KEY 读取密钥
#
# model_info 是必填项, 因为 autogen 需要知道模型的能力特征:
#   - function_calling: 模型能否调用工具/函数
#   - json_output     : 模型能否输出结构化 JSON
#   - structured_output: 模型能否遵循 JSON Schema 输出
#   - vision          : 模型能否理解图片
#   - family          : 模型系列名称 (用于内部路由)
# -----
model_client = OpenAIChatCompletionClient(
    model="qwen-plus",
    api_key=os.getenv("DASHSCOPE_API_KEY"),
    base_url="https://llm-p5jtn7y87smzjpwo.cn-
beijing.maas.aliyuncs.com/compatible-mode/v1",
    model_info={
        "vision": False,          # qwen-plus 在此配置下不支持视觉理解
        "function_calling": True, # 支持函数调用 (工具调用)
        "json_output": True,      # 支持 JSON 格式输出
        "structured_output": True, # 支持结构化输出 (JSON Schema)
        "family": "qwen",         # 模型系列
    },

```

```

)

# -----
# 【知识点 3】AssistantAgent -- 单个智能体
# -----
# 每个 Agent 由两个核心要素定义：
#   1) name (名称)           : Agent 的身份标识，在对话中显示为发言者
#                               注意：只能是英文字母、数字、下划线、连字符
#                               中文名称会导致 autogen 内部校验失败！
#
#   2) system_message (系统提示词)：相当于给 Agent 设定的人设或岗位职责
#       - 它会被放在每条发送给 LLM 的消息最前面 (role: system)
#       - LLM 会根据这个提示词调整回答的语气、视角、风格
#       - 越具体越好，比如你是一名资深 Python 工程师 比 你是程序员 效果更好
#
#   3) model_client (模型客户端)：指定该 Agent 使用哪个 LLM
# -----

# 角色 1：代码阅读者
# 职责：通读代码，理解它在做什么，用简洁的语言描述出来
# 类比：新员工入职后先看代码，写一份代码读懂笔记给同事
code_reader = AssistantAgent(
    name="CodeReader", # 英文命名！autogen 0.7.x 不允许中文名
    model_client=model_client,
    system_message=(
        "你是一名资深 Python 工程师，角色是【代码阅读者】。\\n"
        "你的职责：\\n"
        "1. 仔细阅读用户提供的代码\\n"
        "2. 用简洁的中文解释这段代码的功能和逻辑\\n"
        "3. 指出代码中使用的关键函数、变量和数据流向\\n"
        "4. 每次发言控制在 150 字以内\\n"
        "你的输出将作为后续 Bug 查找的基础，请务必准确、全面。"
    ),
)

# 角色 2：Bug 猎人
# 职责：基于代码阅读者的分析，专门挑毛病—逻辑错误、边界情况、安全隐患等
# 类比：QA 测试员，专门找代码的漏洞和缺陷
bug_finder = AssistantAgent(
    name="BugFinder",
    model_client=model_client,
    system_message=(
        "你是一名资深测试工程师，角色是【Bug 猎人】。\\n"
        "你的职责：\\n"
        "1. 阅读代码阅读者的分析\\n"
        "2. 找出代码中的 Bug、逻辑错误和潜在问题\\n"
        "3. 特别关注：边界情况（如输入为 0、负数、None、极大值等）\\n"
        "4. 每个问题都说明【问题描述】+【可能导致什么后果】\\n"
        "5. 每次发言控制在 200 字以内\\n"
        "如果代码没有问题，也要明确说出未发现 Bug。"
    ),
)

```

```

# 角色 3: 审查总结者
# 职责: 汇总前两位的意见, 给出最终结论和改进建议
# 类比: 技术主管做 code review 总结
reviewer = AssistantAgent(
    name="Reviewer",
    model_client=model_client,
    system_message=(
        "你是一名技术主管, 角色是【审查总结者】。\\n"
        "你的职责: \\n"
        "1. 阅读代码阅读者的分析和 Bug 猎人的发现\\n"
        "2. 给出结构化的审查结论: \\n"
        "    【代码功能】代码阅读者总结的功能\\n"
        "    【发现问题】Bug 猎人找到的问题列表\\n"
        "    【改进建议】针对每个问题的具体修复方案 (给出代码示例) \\n"
        "    【总体评价】代码质量评分 (A/B/C/D) 及理由\\n"
        "3. 每次发言控制在 300 字以内\\n"
        "你的输出是本次审查的最终结论。"
    ),
)

# -----
# 【知识点 4】RoundRobinGroupChat -- 轮转群聊 (多 Agent 协作的核心)
# -----
# RoundRobinGroupChat 让多个 Agent 按顺序轮流发言, 形成一个讨论组。
#
# 工作流程 (以 3 个 Agent、max_messages=6 为例):
#
# 第 1 轮: 用户发起任务 --> CodeReader 回应 (阅读代码)
# 第 2 轮: CodeReader 的输出 --> BugFinder 回应 (找问题)
# 第 3 轮: BugFinder 的输出 --> Reviewer 回应 (总结结论)
# 第 4 轮: Reviewer 的总结 --> CodeReader 补充...
# ...
# 第 6 轮: 达到 max_messages 上限 --> 自动终止
#
# 关键理解:
# - 每个 Agent 都能看到前面所有 Agent 的发言 (共享上下文)
# - 发言顺序就是创建 team 时传入的 Agent 列表顺序
# - termination_condition 决定何时结束对话
#   * MaxMessageTermination: 达到消息条数就停 (最常用)
#   * TextMentionTermination: 出现特定关键词就停
#   * 多个条件可以组合 (OrTermination / AndTermination)
# -----

# 创建轮转群聊团队: 3 个 Agent 按 CodeReader --> BugFinder --> Reviewer 顺序轮流发言
team = RoundRobinGroupChat(
    [code_reader, bug_finder, reviewer],
    termination_condition=MaxMessageTermination(max_messages=6),
    # max_messages=6 的含义:
    # 总共最多产生 6 条 Agent 回复消息 (不计用户初始消息)
    # 3 个 Agent x 每人最多发言 2 次 = 6 条
    # 如果希望多轮深入讨论, 可以调大这个数字
)

```

```

# -----
# 【知识点 5】异步执行与 Console 输出
# -----
# AutoGen 0.7.x 的对话引擎是异步的 (async/await)，原因：
#   1. LLM API 调用是网络请求，异步可以避免阻塞
#   2. 多个 Agent 可能需要并行处理（在更复杂的编排中）
#
# asyncio.run(main()) 是 Python 标准库提供的异步入口
#
# Console() 的作用：
#   - 实时流式输出对话过程（像看直播一样，不是等全部完成才显示）
#   - 自动格式化每条消息，显示发言者名称
#   - 显示 Token 消耗统计
#
# team.run_stream(task='...') 发起一个任务：
#   - task 参数 = 用户的第一条消息（发给所有 Agent）
#   - 返回一个异步迭代器，Console 消费它来打印输出
# -----
async def main():
    # 待审查的示例代码（故意写了一个 Bug：折扣计算错误）
    code = '''
def calculate_discount(price, discount):
    """计算折后价格。
    price: 原价（浮点数）
    discount: 折扣（0-100 之间的整数，如 20 表示打 8 折）
    返回：折后价格
    """
    return price * discount
'''

    # 任务描述：告诉团队要做什么
    task = (
        f"请审查以下 Python 代码，找出其中的问题并给出改进建议：\n\n"
        f"`python\n{code}`"
    )

    # Console 会实时打印每一条对话消息
    # 输出格式示例：
    #   ----- TextMessage (CodeReader) -----
    #   这段代码的功能是计算折后价格...
    #
    #   ----- TextMessage (BugFinder) -----
    #   我发现以下问题：
    #   1. 折扣计算逻辑错误...
    await Console(team.run_stream(task=task))

# -----
# 程序入口
# -----
if __name__ == "__main__":
    print("=" * 60)
    print("多 Agent 协作代码审查开始")

```

```
print("=" * 60)
print()
asyncio.run(main())
print()
print("=" * 60)
print(" 审查完成")
print("=" * 60)
```

工作流程：

[第 1 轮] 代码阅读者：这段代码计算折扣，输入 price 和 discount，输出折扣价
 [第 2 轮] Bug 猎人：发现问题！没有类型检查，discount 可能大于 1
 [第 3 轮] 审查总结者：汇总意见，建议添加类型检查和边界条件验证
 [第 4 轮] 代码阅读者：同意，还需要添加文档字符串
 [第 5 轮] Bug 猎人：补充一点，应该处理负数的情况
 [第 6 轮] 审查总结者：最终结论：代码逻辑正确，但缺少安全检查，建议修改

 **记住：**多 Agent 协作的核心是“分工”——每个 Agent 专注做好一件事，通过对话协作完成任务。

8.5 AutoGen 进阶：人类参与对话 🧡

场景：让 2 个 AI Agent 和 1 个人类一起讨论“如何提高编程效率”。

核心知识点：

- **UserProxyAgent**：代表人类参与对话的 Agent
- **human_input_mode**：控制人类参与的时机
 - **ALWAYS**：每次都问人类
 - **TERMINATE**：只在结束时间问
 - **NEVER**：不问人类（全自动）

完整代码：

```
"""
=====
AutoGen 人机协作：2 个 AI + 1 个人类一起讨论
=====

【知识点 1】UserProxyAgent -- 人类代理
- 作用：让人类参与到 Agent 对话中
- 应用场景：需要人类审核、决策、提供信息的场景
- 类比：会议中的“人类代表”，AI 们讨论到关键点时问他意见

【知识点 2】human_input_mode -- 人类参与模式
- ALWAYS   ：每次都问人类（像会议主持人，每轮都发言）
- TERMINATE：只在对话结束时问（像老板，最后拍板）
- NEVER    ：不问人类（全自动，人类只看不说）
```

【知识点 3】RoundRobinGroupChat -- 轮转群聊

- 多个 Agent 按顺序轮流发言
- 人类代理也可以加入轮转
- 适合"讨论"场景

"""

```
import asyncio
import os
import sys
import io

# 设置 UTF-8 编码 (解决 Windows 中文乱码)
sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8')

from dotenv import load_dotenv

load_dotenv()

from autogen_agentchat.agents import AssistantAgent
from autogen_agentchat.conditions import MaxMessageTermination,
TextMessageTermination
from autogen_agentchat.teams import RoundRobinGroupChat
from autogen_agentchat.ui import Console
from autogen_ext.models.openai import OpenAIChatCompletionClient

# — 配置 LLM (通义千问) —————
model_client = OpenAIChatCompletionClient(
    model="qwen-plus",
    api_key=os.getenv("DASHSCOPE_API_KEY"),
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1",
    model_info={
        "vision": False,
        "function_calling": True,
        "json_output": True,
        "structured_output": True,
        "family": "qwen",
    },
)

# — 创建 AI Agent 1: 效率专家 —————
# 角色: 专门研究如何提高编程效率的专家
efficiency_expert = AssistantAgent(
    name="EfficiencyExpert",
    model_client=model_client,
    system_message=(
        "你是一位编程效率专家, 专门研究如何提高程序员的编程效率。\\n"
        "你的职责:\\n"
        "1. 提出提高编程效率的方法和工具\\n"
        "2. 分享最佳实践和经验\\n"
        "3. 每次发言不超过 100 字\\n"
        "4. 说完后等其他人发言, 如果有人骂你, 你会主动骂他"
    ),
)
```

```

# — 创建 AI Agent 2: 程序员代表 —————
# 角色：代表一线程序员，提出实际工作中的痛点
programmer = AssistantAgent(
    name="Programmer",
    model_client=model_client,
    system_message=(
        "你是一位有 5 年经验的一线程序员。\\n"
        "你的职责：\\n"
        "1. 提出程序员在日常工作中遇到的效率问题\\n"
        "2. 反馈哪些工具和方法实际好用，哪些不好用\\n"
        "3. 每次发言不超过 100 字\\n"
        "4. 说完后等其他人员发言"
    ),
)

# — 创建人类代理 —————
# 【关键】UserProxyAgent 让人类参与到对话中
#
# human_input_mode="ALWAYS" 的含义：
# - 每次轮到人类发言时，都会暂停等待人类输入
# - 人类可以输入文字，也可以按回车跳过
# - 适合需要人类全程参与的场景
#
# 其他选项：
# - "TERMINATE"：只在对话结束时问一次
# - "NEVER"：不问人类（但这里我们用 ALWAYS）
from autogen_agentchat.agents import UserProxyAgent

human = UserProxyAgent(
    name="HumanUser",
    input_func=input, # 使用标准 input() 函数获取人类输入
)

# — 创建轮转群聊 —————
# 【关键】把人类代理加入到群聊中
#
# 发言顺序：效率专家 → 程序员代表 → 人类参与者 → 效率专家 → ...
#
# 终止条件：
# - MaxMessageTermination(15)：最多 15 条消息
# - TextMessageTermination(["人类参与者"])：人类说"结束"时终止
# - 两个条件满足任意一个就终止（用 | 组合）
team = RoundRobinGroupChat(
    [efficiency_expert, programmer, human],
    termination_condition=MaxMessageTermination(15) |
    TextMessageTermination(["HumanUser"]),
)

# — 主函数 —————
async def main():
    print("\\n" + "=" * 60)
    print("  人机协作讨论：如何提高编程效率")

```

```

print("=" * 60)
print("\n📋 讨论规则：")
print(" - 效率专家和程序员代表会轮流发言")
print(" - 每次轮到你时，你可以输入观点，或按回车跳过")
print(" - 输入'结束'可以提前结束讨论")
print("\n开始讨论...\n")

# 启动讨论
# task 参数：讨论的主题
await Console(
    team.run_stream(
        task="请大家讨论：如何提高编程效率？每人分享 1-2 个方法。"
    )
)

print("\n" + "=" * 60)
print(" 讨论结束")
print("=" * 60)

# — 程序入口 —————
if __name__ == "__main__":
    asyncio.run(main())

```

运行效果（示例）：

```

=====
人机协作讨论：如何提高编程效率
=====

📋 讨论规则：
- 效率专家和程序员代表会轮流发言
- 每次轮到你时，你可以输入观点，或按回车跳过
- 输入'结束'可以提前结束讨论

开始讨论...

----- TextMessage (效率专家) -----
提高编程效率的首要方法是使用 AI 编程助手，如 Copilot 或 Cursor。
它们可以自动生成代码、补全代码片段，节省大量时间。

----- TextMessage (程序员代表) -----
同意。但我发现 AI 助手生成的代码有时需要大量修改，反而浪费时间。
关键是要学会写好提示词（Prompt），让 AI 生成更准确的代码。

----- TextMessage (人类参与者) -----
我觉得还需要好的开发环境。我用 VS Code + 插件，效率提升很大。
你们有什么推荐的插件吗？

----- TextMessage (效率专家) -----
推荐几个 VS Code 插件：
1. GitLens：查看代码历史和 blame

```

2. Prettier：自动格式化代码
3. Error Lens：直接在代码行显示错误信息

----- TextMessage（程序员代表） -----

我再加一个：Thunder Client，直接在 VS Code 里测试 API，不用切换到 Postman，很方便。

----- TextMessage（人类参与者） -----

好的，我试试这些插件。还有关于代码审查的建议吗？

----- TextMessage（效率专家） -----

代码审查可以用自动化工具，如 SonarQube 或 CodeClimate。它们可以自动检查代码质量、安全漏洞，减少人工审查的工作量。

----- TextMessage（程序员代表） -----

但自动化工具不能检查业务逻辑。我建议：

1. 自动化工具检查代码风格和安全问题
2. 人工审查业务逻辑和架构设计
3. 两者结合效果最好

----- TextMessage（人类参与者） -----

明白了。我总结一下今天的讨论：

1. 使用 AI 编程助手，但要写好提示词
2. 配置好开发环境和插件
3. 代码审查用自动化工具 + 人工审查

谢谢大家！

----- TextMessage（人类参与者） -----

结束

```
=====
    讨论结束
=====
```

代码解析：

1. **UserProxyAgent**：让人类参与到 Agent 对话中

```
human = UserProxyAgent(
    name="人类参与者",
    input_func=input, # 使用标准 input() 函数
)
```

2. **human_input_mode**：控制人类参与的时机

```
# "ALWAYS": 每次都问人类
# "TERMINATE": 只在结束时间问
# "NEVER": 不问人类
```

3. 把人类加入群聊：

```
team = RoundRobinGroupChat(
    [efficiency_expert, programmer, human], # 人类也在列表中
    ...
)
```

4. 终止条件组合：

```
termination_condition=MaxMessageTermination(15) | TextMessageTermination(["人类参与者"])
# 达到 15 条消息，或人类说"结束"，都会终止
```

💡 **记住：**AutoGen 的人机协作是它的核心优势——让人类在关键决策点参与，既发挥 AI 的效率，又保留人类的判断力。

8.6 CrewAI：基于角色的多 Agent 框架

CrewAI 是一个基于**角色扮演**的多 Agent 框架，特点是**简单、直观、像组建团队一样**。

核心思想：

把每个 Agent 想象成一个“团队成员”，每个成员有自己的**角色（Role）、目标（Goal）和背景（Backstory）**，然后给每个成员分配**任务（Task）**，最后让他们按顺序或并行完成任务。

CrewAI 的核心概念：

概念	类比	说明
Agent	团队成员	有角色、目标、背景的 AI
Task	工作任务	Agent 要完成的具体任务
Crew	团队	多个 Agent 组成的协作团队
Process	工作流程	任务的执行顺序（顺序 / 层级）

AutoGen vs CrewAI：

维度	AutoGen	CrewAI
核心概念	对话	角色 + 任务

维度	AutoGen	CrewAI
协作方式	Agent 之间对话	按流程执行任务
适用场景	需要灵活对话	需要明确分工
学习曲线	较陡	较平缓
代码量	较多	较少

安装：

```
pip install crewai
```

8.7 CrewAI 实战：写技术博客

场景：3 个 Agent 组成内容创作团队，写一篇技术博客。

完整代码：

```
"""
CrewAI 实战：写技术博客
安装依赖：pip install crewai

运行前准备：
1. 设置环境变量 DASHSCOPE_API_KEY（阿里云百炼 API Key）
2. 确保已安装依赖：pip install python-dotenv
"""

# — 导入 CrewAI 核心组件 —————
from crewai import Agent, Task, Crew, Process

# — 配置 LLM：使用 Qwen（阿里云 DashScope） —————
from crewai.llm import LLM
import os
from dotenv import load_dotenv

load_dotenv()

llm = LLM(
    model="qwen-plus",
    api_key=os.getenv("DASHSCOPE_API_KEY"),
    base_url="https://llm-p5jtn7y87smzjpwo.cn-beijing.maas.aliyuncs.com/compatible-mode/v1",
)

# — 创建 Agent 1：研究员 —————
# role: 角色（这个 Agent 是干什么的）
# goal: 目标（这个 Agent 要达成什么）
```

```
# backstory: 背景 (这个 Agent 的经历, 影响它的行为方式)
researcher = Agent(
    role="AI 技术研究员",
    goal="收集和整理 AI 领域的最新进展和重要论文",
    backstory="""你是一位资深的 AI 技术研究员, 有 10 年研究经验。
你擅长从海量信息中筛选出最有价值的内容, 并用通俗易懂的语言解释复杂概念。
你特别关注大语言模型、Agent、RAG 等前沿技术。""",
    verbose=True, # 开启详细输出, 可以看到 Agent 的思考过程
    allow_delegation=False, # 不允许委派任务给其他 Agent
    llm=llm, # 使用 Qwen
)

# — 创建 Agent 2: 写手 —————
writer = Agent(
    role="技术博客写手",
    goal="根据研究资料, 撰写高质量的技术博客文章",
    backstory="""你是一位技术博客写手, 擅长把复杂的技术概念用通俗的语言写出来。
你的文章风格: 清晰、有趣、有深度, 深受开发者喜爱。
你特别擅长写教程类文章, 能让读者快速上手。""",
    verbose=True,
    allow_delegation=False,
    llm=llm,
)

# — 创建 Agent 3: 校对 —————
editor = Agent(
    role="内容编辑",
    goal="检查文章的逻辑、语法和可读性, 确保文章质量",
    backstory="""你是一位资深内容编辑, 有 15 年编辑经验。
你擅长发现文章中的逻辑漏洞、语法错误和表达不清的地方。
你对文章质量要求很高, 但也懂得平衡专业性和可读性。""",
    verbose=True,
    allow_delegation=False,
    llm=llm,
)

# — 创建 Task 1: 研究任务 —————
# description: 任务描述 (要做什么)
# expected_output: 期望的输出 (要输出什么)
# agent: 执行任务的 Agent
research_task = Task(
    description="""研究 AI 领域的最新进展, 重点关注:
1. 大语言模型的最新发展 (如 GPT-4、Claude、Qwen)
2. Agent 技术的最新突破
3. RAG (检索增强生成) 的应用场景
请整理成结构化的研究报告。""",
    expected_output="""一份结构化的研究报告, 包含:
- 3-5 个重要进展
- 每个进展的简要说明 (100 字以内)
- 相关论文或项目的链接 (如果有) """,
    agent=researcher, # 由研究员执行
)

# — 创建 Task 2: 写作任务 —————
```

```

write_task = Task(
    description="""根据研究报告，撰写一篇技术博客文章。
    要求：
    1. 标题吸引人
    2. 开头用一个有趣的故事或案例引入
    3. 正文分 3-4 个小节，每节讲一个重点
    4. 结尾总结要点，给出行动建议
    5. 字数 1500-2000 字""",
    expected_output="""一篇完整的技术博客文章，包含：
    - 标题
    - 引言（故事/案例）
    - 正文（3-4 个小节）
    - 结论（总结 + 行动建议）""",
    agent=writer, # 由写手执行
    context=[research_task], # 依赖研究任务的输出
)

# — 创建 Task 3: 校对任务 —————
edit_task = Task(
    description="""检查文章的质量，重点关注：
    1. 逻辑是否清晰
    2. 语法是否正确
    3. 表达是否通俗易懂
    4. 是否有事实错误
    请给出修改建议。""",
    expected_output="""一份校对报告，包含：
    - 文章评分（1-10 分）
    - 发现的问题列表
    - 修改建议
    - 最终版本（如果问题不大，直接修改）""",
    agent=editor, # 由编辑执行
    context=[write_task], # 依赖写作任务的输出
)

# — 创建 Crew（团队） —————
# agents: 团队成员列表
# tasks: 任务列表（按顺序执行）
# process: 执行流程（sequential = 顺序执行，hierarchical = 层级执行）
# verbose: 是否输出详细日志
crew = Crew(
    agents=[researcher, writer, editor], # 团队成员
    tasks=[research_task, write_task, edit_task], # 任务列表
    process=Process.sequential, # 顺序执行：研究 → 写作 → 校对
    verbose=True,
)

# — 启动团队 —————
if __name__ == "__main__":
    print("🚀 开始创作技术博客...\n")

    # 启动团队，执行任务
    result = crew.kickoff()

    print("\n" + "=" * 50)

```

```
print("✅ 创作完成！")
print("=" * 50)
print("\n最终结果：")
print(result)
```

运行效果（示例）：

🚀 开始创作技术博客...

[研究员] 正在研究 AI 领域的最新进展...

[研究员] 任务完成！输出：研究报告...

[写手] 正在根据研究报告撰写文章...

[写手] 任务完成！输出：博客文章...

[编辑] 正在检查文章质量...

[编辑] 任务完成！输出：校对报告...

=====

✅ 创作完成！

=====

最终结果：

[编辑的校对报告 + 最终文章]

代码解析：

1. **创建 Agent**：每个 Agent 有 **role**（角色）、**goal**（目标）、**backstory**（背景）
2. **创建 Task**：每个 Task 有 **description**（描述）、**expected_output**（期望输出）、**agent**（执行者）
3. **创建 Crew**：把 Agent 和 Task 组合成团队
4. **启动团队**：**crew.kickoff()** 开始执行任务

💡 **记住**：CrewAI 的核心是“角色 + 任务”——每个 Agent 有自己的角色和目标，按顺序完成任务。

8.8 CrewAI 实战：旅行规划团队

场景：3 个 Agent 组成旅行规划团队，帮你规划一次旅行。

完整代码：

```
"""
CrewAI 实战：旅行规划团队
安装依赖：pip install crewai
"""

from crewai import Agent, Task, Crew, Process
from crewai.llm import LLM
import os
```

```

from dotenv import load_dotenv

load_dotenv()

llm = LLM(
    model="qwen-plus",
    api_key=os.getenv("DASHSCOPE_API_KEY"),
    base_url="https://llm-p5jtn7y87smzjpwo.cn-
beijing.maas.aliyuncs.com/compatible-mode/v1",
)

# — 创建 Agent 1: 旅行顾问 —————
travel_advisor = Agent(
    role="资深旅行顾问",
    goal="根据用户需求，推荐最佳旅行目的地和时间",
    backstory="""你是一位资深旅行顾问，去过 50 多个国家。
你擅长根据用户的预算、兴趣和时间，推荐最合适的旅行方案。
你特别了解亚洲和欧洲的旅游路线。""",
    verbose=True,
    allow_delegation=False,
    llm=llm,
)

# — 创建 Agent 2: 预算规划师 —————
budget_planner = Agent(
    role="旅行预算规划师",
    goal="制定详细的旅行预算，包括机票、酒店、餐饮、景点门票",
    backstory="""你是一位专业的旅行预算规划师，擅长控制旅行成本。
你能在保证旅行质量的前提下，帮用户省钱。
你特别擅长找性价比高的酒店和机票。""",
    verbose=True,
    allow_delegation=False,
    llm=llm,
)

# — 创建 Agent 3: 行程规划师 —————
itinerary_planner = Agent(
    role="行程规划师",
    goal="制定详细的每日行程安排，包括景点、餐厅、交通",
    backstory="""你是一位经验丰富的行程规划师，擅长安排紧凑但不累的行程。
你特别了解各个景点的开放时间和最佳游览顺序。
你能根据用户的体力和兴趣，安排最合理的行程。""",
    verbose=True,
    allow_delegation=False,
    llm=llm,
)

# — 创建 Task 1: 推荐目的地 —————
recommend_task = Task(
    description="""用户需求：
- 目的地：日本
- 时间：5 天
- 预算：10000 元
- 兴趣：美食、文化、购物

```

```

请推荐最佳的旅行路线和必去的景点。"""，
    expected_output="""一份旅行推荐报告，包含：
- 推荐的城市和景点（3-5 个）
- 每个景点的亮点（50 字以内）
- 建议的游览顺序"""，
    agent=travel_advisor,
)

# — 创建 Task 2: 制定预算 —————
budget_task = Task(
    description="""根据旅行推荐，制定详细的预算方案。
包括：
1. 机票（往返）
2. 酒店（4 晚）
3. 餐饮（5 天）
4. 景点门票
5. 交通（市内）
6. 购物预算

总预算控制在 10000 元以内。"""，
    expected_output="""一份详细的预算表，包含：
- 各项费用的明细
- 总计金额
- 省钱建议"""，
    agent=budget_planner,
    context=[recommend_task], # 依赖推荐任务的输出
)

# — 创建 Task 3: 安排行程 —————
itinerary_task = Task(
    description="""根据推荐和预算，制定详细的 5 天行程。
包括：
1. 每天的景点安排
2. 餐厅推荐（午餐、晚餐）
3. 交通方式
4. 注意事项

要求：行程紧凑但不累，每天步行不超过 15000 步。"""，
    expected_output="""一份详细的行程安排表，包含：
- 5 天的详细行程（每天 3-4 个活动）
- 每个活动的时间、地点、交通方式
- 餐厅推荐（人均消费）
- 注意事项"""，
    agent=itinerary_planner,
    context=[recommend_task, budget_task], # 依赖前两个任务
)

# — 创建团队并启动 —————
crew = Crew(
    agents=[travel_advisor, budget_planner, itinerary_planner],
    tasks=[recommend_task, budget_task, itinerary_task],
    process=Process.sequential,
    verbose=True,

```

```
)

if __name__ == "__main__":
    print("✈️ 开始规划日本之旅...\n")
    result = crew.kickoff()

    print("\n" + "=" * 50)
    print("✅ 旅行规划完成! ")
    print("=" * 50)
    print("\n最终行程: ")
    print(result)
```

运行效果（示例）：

```
✈️ 开始规划日本之旅...

[旅行顾问] 正在推荐目的地和景点...
[旅行顾问] 推荐：东京、京都、大阪...

[预算规划师] 正在制定预算...
[预算规划师] 预算明细：机票 3000 元，酒店 2500 元...

[行程规划师] 正在安排行程...
[行程规划师] Day 1：东京浅草寺、上野公园...

=====
✅ 旅行规划完成!
=====

最终行程：
[详细的 5 天行程安排]
```

8.9 CrewAI 进阶：层级执行流程

CrewAI 支持两种执行流程：

流程	说明	适用场景
sequential	顺序执行，一个任务接一个	任务有依赖关系
hierarchical	层级执行，有一个"经理"Agent 协调	需要动态决策

层级执行示例：

```
"""
CrewAI 层级执行：经理 Agent 协调团队
"""

from crewai import Agent, Task, Crew, Process
```

```
from crewai.llm import LLM
import os
from dotenv import load_dotenv

load_dotenv()

llm = LLM(
    model="qwen-plus",
    api_key=os.getenv("DASHSCOPE_API_KEY"),
    base_url="https://llm-p5jtn7y87smzjpwo.cn-beijing.maas.aliyuncs.com/compatible-mode/v1",
)

# — 创建经理 Agent —————
manager = Agent(
    role="项目经理",
    goal="协调团队，确保任务高质量完成",
    backstory="""你是一位经验丰富的项目经理，擅长协调团队。
你能根据团队成员的能力，合理分配任务。
你会监控任务进度，及时处理问题。""",
    verbose=True,
    allow_delegation=True, # 允许委派任务
    llm=llm,
)

# — 创建团队成员 —————
researcher = Agent(
    role="研究员",
    goal="收集资料",
    backstory="你是研究员，擅长收集资料",
    verbose=True,
    llm=llm,
)

writer = Agent(
    role="写手",
    goal="写文章",
    backstory="你是写手，擅长写文章",
    verbose=True,
    llm=llm,
)

# — 创建任务 —————
research_task = Task(
    description="研究 AI 最新进展",
    expected_output="研究报告",
    agent=researcher,
)

write_task = Task(
    description="写技术博客",
    expected_output="博客文章",
    agent=writer,
)
```

```
# — 创建团队（层级执行） —————
crew = Crew(
    agents=[researcher, writer, manager],
    tasks=[research_task, write_task],
    process=Process.hierarchical, # 层级执行
    manager_agent=manager, # 指定经理 Agent
    verbose=True,
)

if __name__ == "__main__":
    print("🚀 开始层级执行...\n")
    result = crew.kickoff()
    print("\n最终结果：")
    print(result)
```

层级执行的工作流程：



💡 记住：

- **sequential** = 固定流程，像流水线
- **hierarchical** = 经理协调，像团队管理

8.10 AutoGen vs CrewAI：如何选择？

场景	推荐框架	原因
需要灵活对话	AutoGen	Agent 之间可以自由对话
需要明确分工	CrewAI	每个 Agent 有明确的角色和任务
任务有依赖	CrewAI	支持任务依赖（context）
需要人机协作	AutoGen	更容易让人类参与对话
快速原型	CrewAI	代码量少，上手快
复杂对话流	AutoGen	对话流更灵活

实际项目中：

- 如果只是"对话协作"（如讲相声、讨论问题）→ **AutoGen**
- 如果是"任务分工"（如写文章、做项目）→ **CrewAI**
- 如果两者都需要 → 可以混合使用

```
import os
import sys
import io
import json

sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8')

from openai import OpenAI
from dotenv import load_dotenv
load_dotenv()

client = OpenAI(
    api_key=os.getenv("DASHSCOPE_API_KEY"),
    base_url="https://llm-p5jtn7y87smzjpwo.cn-beijing.maas.aliyuncs.com/compatible-mode/v1",
)

# — 工具实现 —————
def search_weather(city: str) -> str:
    """查询指定城市的天气"""
    return f"{city}明天晴, 15-25°C"

def book_flight(origin: str, destination: str, date: str) -> str:
    """预订从 origin 到 destination 的机票, 日期为 date"""
    return f"已订票: {origin}→{destination}, {date}"

# — 工具定义 (告诉 LLM 有哪些工具可用) —————
TOOLS = [
    {
        "type": "function",
        "function": {
            "name": "search_weather",
            "description": "查询指定城市未来的天气情况",
            "parameters": {
                "type": "object",
                "properties": {
                    "city": {"type": "string", "description": "城市名称, 如 '北京', '上海'"}
                }
            },
            "required": ["city"],
        },
    },
]
```

```

{
    "type": "function",
    "function": {
        "name": "book_flight",
        "description": "预订两个城市之间的航班",
        "parameters": {
            "type": "object",
            "properties": {
                "origin": {"type": "string", "description": "出发城市"},
                "destination": {"type": "string", "description": "目的城市"},
                "date": {"type": "string", "description": "出发日期, 如 '明天',
'2024-03-15'"},
            },
            "required": ["origin", "destination", "date"],
        },
    },
},
],

# 工具名称 → 实际函数的映射
AVAILABLE_FUNCTIONS = {
    "search_weather": search_weather,
    "book_flight": book_flight,
}

# — Agent 主循环 (基于 function calling) —————
def agent_loop(user_input: str):
    system_prompt = (
        "你是一个差旅助手, 帮助用户规划出差行程。\\n"
        "你可以使用提供的工具查询天气和订票。\\n"
        "在给出最终建议之前, 请先调用工具获取真实信息。\\n"
        "获取所有必要信息后, 给出完整的行程安排。"
    )

    messages = [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_input},
    ]

    while True:
        response = client.chat.completions.create(
            model="qwen-plus",
            messages=messages,
            tools=TOOLS,
        )

        choice = response.choices[0]
        message = choice.message

        # — 情况 1: LLM 选择调用工具 —————
        if message.tool_calls:
            for tool_call in message.tool_calls:
                func_name = tool_call.function.name

```

```

func_args = json.loads(tool_call.function.arguments)
func = AVAILABLE_FUNCTIONS[func_name]

print(f"[调用工具] {func_name}({json.dumps(func_args,
ensure_ascii=False)})")
result = func(**func_args)
print(f"[工具返回] {result}")

# 将工具调用和结果追加到上下文
messages.append(message) # assistant 的 tool_calls 消息
messages.append({
    "role": "tool",
    "tool_call_id": tool_call.id,
    "content": str(result),
})

# — 情况 2: LLM 回复文本（不再需要工具） —————
elif message.content:
    print(f"\n{message.content}")
    break

# — 运行 —————
agent_loop("帮我安排明天去北京出差，我从上海出发")

```

 **记住：**框架帮你封装了细节，但核心逻辑都是“思考→行动→观察→循环”。

九、Agent 的挑战与局限

1. 幻觉问题

Agent 可能会“幻觉”——调用错误的工具，或做出错误决策。

示例：用户问“帮我订机票”，Agent 却调用了“订酒店”API。

应对：加强工具描述的准确性，添加人工审核环节（HITL ）。

2. 安全性问题

Agent 有调用工具的能力，如果被恶意利用，可能造成实际损失。

示例：恶意用户让 Agent “帮我删除所有订单”，Agent 真的执行了。

应对：添加权限控制，敏感操作需要人工确认。

3. 成本问题

Agent 每次“思考”都要调用 LLM，多步任务成本高。

示例：安排一次出差，Agent 可能要调用 LLM 10 次以上。

应对：优化流程，减少不必要的思考步骤；使用更便宜的模型处理简单决策。

4. 可控性问题

Agent 是"自主决策"的，有时候不按你的预期走。

示例：你让 Agent "帮我订便宜的机票"，它订了凌晨 3 点的红眼航班。

应对：在 System Prompt 中明确约束，添加"禁止行为"。

5. 调试困难

Agent 是多步循环的，出了问题很难定位。

示例：Agent 最终给出了错误结果，但你不知道是哪一步出了问题。

应对：添加详细日志，记录每一步的思考、行动、观察。

 **记住：**Agent 强大但也有风险。生产环境中一定要添加**人工审核（HITL **），不能完全放手。

十、面试常见问题

Q1：什么是 Agent？和 Prompt 有什么区别？

标准答案：

Agent 是能自主决策 + 调用工具 + 完成任务的 AI。和 Prompt 的区别：

维度	Prompt	Agent
自主性	被动回答	主动决策
工具调用	不能调用工具	能调用外部工具
任务复杂度	单步任务	多步复杂任务
记忆能力	无状态	有记忆（短期+长期）

加分回答：

"Agent 的本质是'LLM + 工具 + 记忆 + 规划'。它不只是'聊天'，而是能'干活'——调用 API、查数据库、发邮件，真正完成任务。"

Q2：Agent 和 Skill、Workflow 有什么区别？

标准答案：

维度	Skill	Workflow	Agent
决策方式	按固定流程	按预定义流程	自主决策
适用场景	流程固定	流程清晰	需要灵活判断

维度	Skill	Workflow	Agent
可控性	高	最高	较低
灵活性	低	低	最高

加分回答：

"Skill 是'你定好规矩，AI 照着做'；Workflow 是'预定义流程，按步骤执行'；Agent 是'你说目标，AI 自己判断怎么做'。实际项目中，经常把三者结合使用。"

Q3：4 种主流开发范式是什么？

标准答案：

- 1. 简单 LLM 应用：直接调用模型 API，只是"聊天"
- 2. 单智能体：一个 Agent 处理所有任务，由模型驱动
- 3. 工作流：预定义流程，确定性强，像流水线
- 4. 多智能体系统：多个 Agent 协作，灵活但难控制

加分回答：

"选择哪种范式取决于场景。简单任务用单智能体；流程明确用工作流；复杂协作用多智能体。实际项目中，经常混合使用——用工作流编排多个 Agent。"

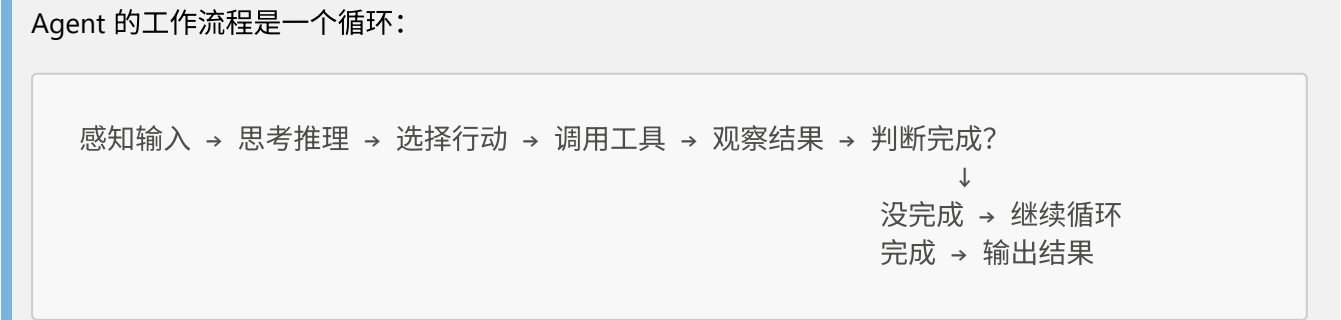
Q4：Agent 的核心组成部分有哪些？

标准答案：

- 四个核心部分：
- 1. LLM（大脑）：负责思考、推理、决策
 - 2. 工具集（Function Calling）：调用外部工具
 - 3. 记忆系统：短期记忆 + 长期记忆
 - 4. 规划器：把复杂任务拆解成多个步骤

Q5：Agent 是怎么工作的？

标准答案：



加分回答：

"这就是 ReAct（Reasoning + Acting）范式 📖——Agent 先推理决定做什么，再行动调用工具，然后观察结果，再推理下一步...循环直到任务完成。"

Q6：单智能体有哪些挑战？

标准答案：

- 四个核心挑战：
- 1. **工具过多**：模型在决策时无所适从
 - 2. **上下文膨胀**：多轮执行后，消息上下文变大，消耗大量 Token
 - 3. **复杂任务**：需要多个步骤，还需要专业领域技能
 - 4. **可维护性**：复杂任务场景下可维护性变差

加分回答：

"这就是为什么需要工作流和多智能体系统——在工程上做更多事情，来弥补模型能力的不足。"

Q7：工作流和多智能体有什么区别？

标准答案：

维度	工作流	多智能体
流程	预定义，确定性	模型驱动，对话式
可控性	高	低
灵活性	低	高
适用场景	流程清晰	任务边界不清晰

加分回答：

"工作流像固定的流水线，强调透明度和安全性；多智能体像团队合作，强调灵活性和适应性。选择哪个取决于场景的确定性和灵活性需求。"

Q8：怎么开发一个 Agent？

标准答案：

- 两种方法：
- 1. **用框架（推荐）**：LangChain、LangGraph、AutoGen 等
 - 2. **手动实现**：理解原理，自己写循环逻辑
- 核心逻辑都是"思考→行动→观察→循环"。

Q9：Agent 有哪些实际应用场景？

标准答案：

常见场景：	
场景	说明
AI 客服	自动处理用户咨询、投诉、退货
编程助手	自动调试、代码审查、测试
数据分析	自动查询数据库、生成报表
办公助手	自动安排会议、发邮件、整理文档
旅行规划	自动订机票、酒店、安排行程

Q10：怎么让 Agent 更可靠？

标准答案：

方法：
1. 添加人工审核（HITL  ）：敏感操作需要人工确认
2. 约束 System Prompt：明确告诉 Agent "不要做什么"
3. 添加工具验证：调用工具前检查参数合法性
4. 添加日志：记录每一步的思考、行动、观察
5. 使用更可靠的 LLM：GPT-4 比 GPT-3.5 更可靠

加分回答：

"还可以用'多 Agent 协作'——一个 Agent 做决策，另一个 Agent 审核。就像公司的'双人复核'制度，降低单点失误的风险。"
 面试技巧： 回答 Agent 相关问题时，先说"是什么"，再说"怎么工作"，最后说"应用场景"。面试官喜欢既懂原理又有实践经验的候选人。

总结一句话

Agent = 能自主决策 + 调用工具 + 完成任务的 AI。从"工具"到"队友"的进化。

 术语对照索引

本章出现位置	术语	术语手册中对应章节
开篇	Agent	三、Agent 架构与协作
第二节	Function Calling	三、Agent 架构与协作
第二节	Skill	三、Agent 架构与协作
第三节	Prompt Engineering	二、提示与交互

本章出现位置	术语	术语手册中对应章节
第四节	Workflow	三、Agent 架构与协作
第四节	Multi-Agent	三、Agent 架构与协作
第五节	Context Engineering	二、提示与交互
第五节	向量数据库	五、知识增强
第八节	LangChain	三、Agent 架构与协作
第八节	LangGraph	三、Agent 架构与协作
第九节	HITL（人在回路）	二、提示与交互
第十节	ReAct	四、推理范式

学完本章你应该认识的术语

- ☐ **Agent** — 是什么？和 Prompt/Skill/Workflow 有什么区别？
- ☐ **Function Calling** — Agent 怎么调用外部工具？
- ☐ **单智能体 / 工作流 / 多智能体** — 4 种开发范式的区别？
- ☐ **ReAct** — Agent 的工作流程是什么？
- ☐ **LangChain / LangGraph** — 怎么开发一个 Agent？
- ☐ **HITL** — 为什么需要人工审核？

 为什么要学习 Agent？

- 简历中的职责就包含大量的 Agent 开发

工作经历

某垂直领域物流科技公司	AI应用工程师	2024.10 - 2026.03
福建海通发展股份有限公司	数据分析师	2024.06 - 2024.10

项目经历

航运智能 Agent 系统	AI应用开发	2024.10 - 2026.03
---------------	--------	-------------------

1. 项目背景：

航运行业涉及物流贸易、船舶航行、监管服务等多环节，存在信息分散、决策依赖经验、风险响应滞后等痛点，且不同角色（货主 / 船东 / 监管部门 / 船员）需求差异显著。本项目立足航运核心业务场景立项，一是借助大模型与 Agent 智能体解放航运全流程人力成本，二是通过智能化能力提升决策效率与风险防控水平，三是依托行业定制化能力形成差异化竞争优势，覆盖政企全链路服务需求
2. 技术栈：LangChain、Agent 智能体、RAG/Embedding/Rerank、LLM、改进的 A * 算法 / 多目标优化算法、FastAPI、MySQL、Milvus、InfluxDB、Neo4j、NLP
3. 核心职责：多角色 Agent 体系构建：针对不同业务端设计定制化 Agent，覆盖物流贸易、船舶航行、航运监管、船员培训四大核心场景。

物流贸易全流程 Agent：整合运价预判、多式联运路径规划、风险预警、智能制单模块，依托 HS 编码数据库与 NLP 技术实现智能制单，联动生成合规单据与风险报告；

船舶航行智能 Agent：融合气象大模型、仿真工具与路径规划算法实现智能气导规划，基于时序数据监控船舶动态，自动识别异常行为并生成事故分析 / 责任判定参考报告；

航运监管 / 船员培训 Agent：对接海事监管数据中台与船员考试题库，构建向量知识库，实现船舶监管预警、事故调查支撑，以及船员智能出题、判题、答疑，为监管政策制定与考试大纲优化提供数据支撑；

全流程技术落地：基于 LangChain 搭建 Agent 核心 workflow，整合多源数据（航运 / 气象 / 政策）与第三方 API，通过 RAG 增强知识检索精度，实现从数据调取、算法计算到报告生成 / 方案输出的端到端智能化。

- 部分岗位职责直接以 agent 开发命名

BOSS直聘



Q

筛选

22-35K

Python



地铁5号线直达 南山区 科技园 ✕

30-35K

本科



地铁51分钟 福田区 车公庙 ×

20-40K

Python



方锦旗 · 运营总监

地铁57分钟 龙岗区 坂田 ×

Agent开发工程师 【猎头岗】

25-50K

北京易立纵横企业... 未融资 20-99人

- 1-3年
- 本科
- 国内院校优先
- Python
- Scrapy



薛银钟 · HR


职位


有了


消息


我的